EEP 524
Applied High-Performance GPU Computing

LECTURE 3 : Tuesday, October 20, 2026

Instructor: Dr. Colin Reinhardt

University of Washington - Professional Masters Program

Autumn 2026

Lecture 3 : Outline

- GPU SIMD/SIMT and Warp concepts
- Parallel performance theory, metrics, and measures
- Basic Matrix Multiplication (MMULT) on GPU
  - algorithm and kernel
- GPU HW Architecture - Device memory hierarchy
  - Improving the MM kernel performance
- Tiled parallel MMULT algorithm

- Overview of HW2

HW2 due Tue, 27 OCT by 5:59 PM

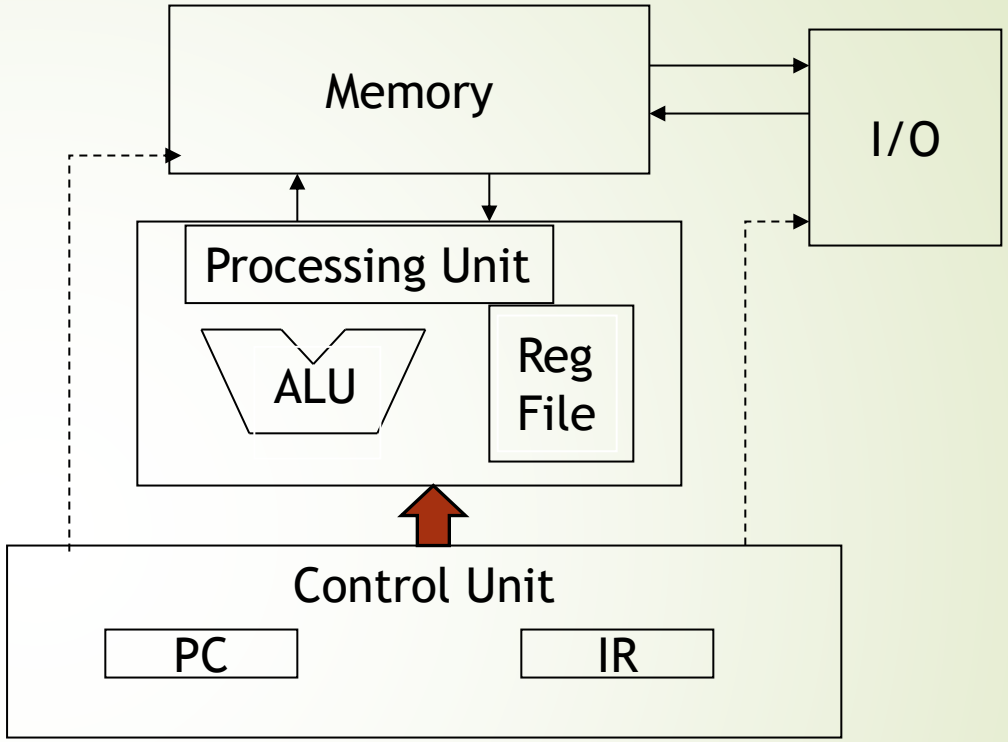
Class Logistics

- **REMINDER**: AppStream sessions expire in 4 hrs so save all work to a persistent (cloud) location beforehand!

- TA office hours are active!
  - Devika – Thursdays
  - Diya – Mondays 630-830 PM
- HW2 procedures will be published after lecture
- **HW reminder**: for full credit responses to the multiple choice questions should include the answer as well as the explanation
- Final project overview/details will be in L5

- READING SET updated - see Pages/Reading Sets

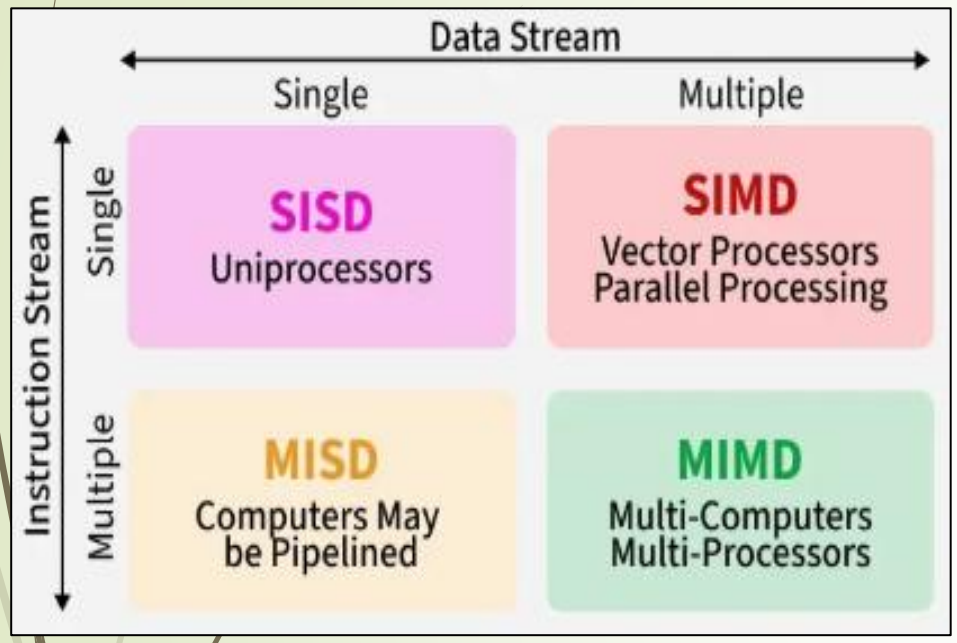
The Von-Neumann Model
computer architecture



Key Components

1. A processing unit with both an arithmetic logic unit (ALU) and processor registers
2. A control unit that includes an instruction register (IR) and a program counter (PC)
3. Memory that stores both data and instructions in same address space
4. External mass storage
5. Input and output mechanism

Flynn's Taxonomy

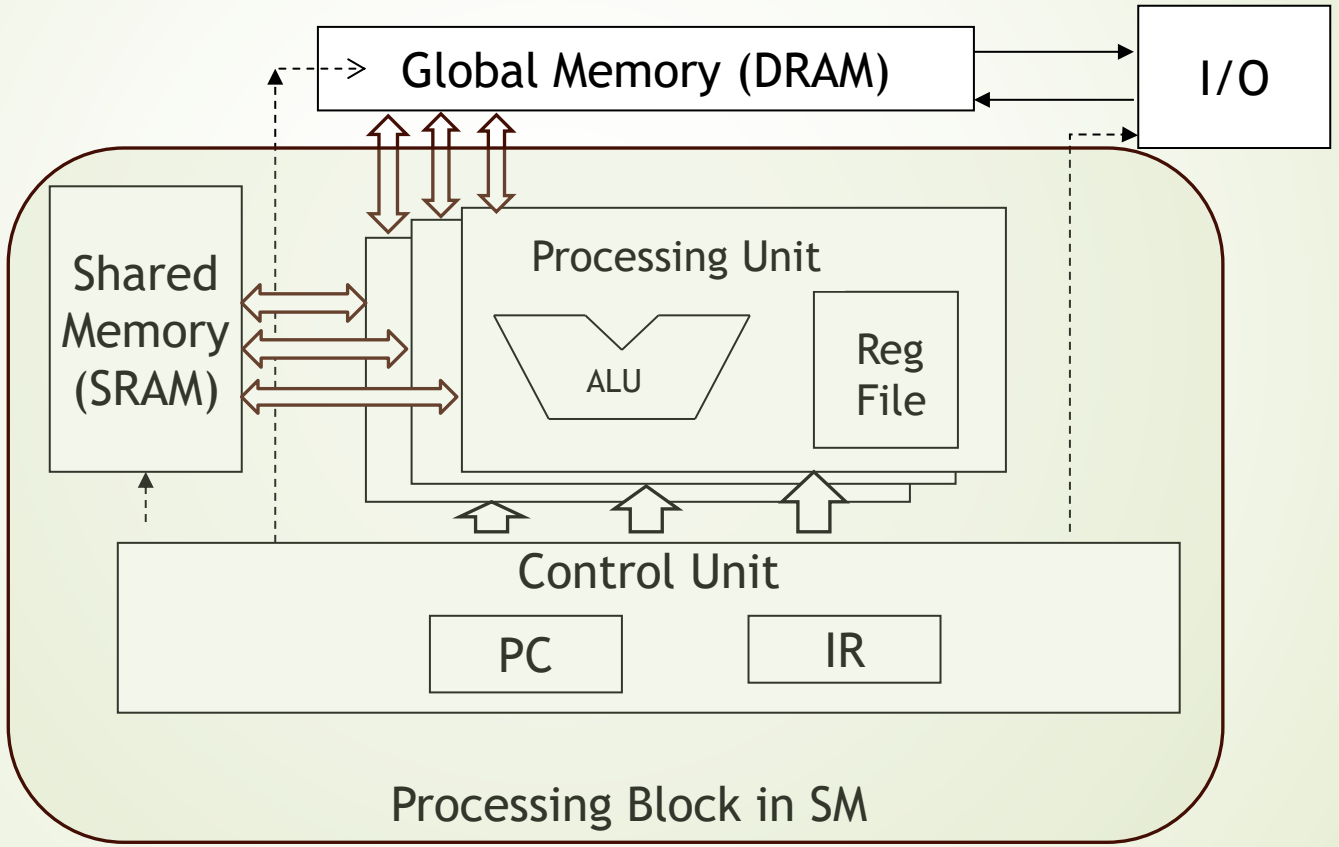


Classifications of Computer Architecture
Dr/Professor Michael J. Flynn, Stanford, 1966

- SISD (Single Instruction, Single Data)
- SIMD (Single Instruction, Multiple Data)
- MISD (Multiple Instruction, Single Data)
- MIMD (Single Instruction, Multiple Data)

The Von-Neumann Model with **SIMD** units

Single Instruction Multiple Data: (SIMD)



SIMD versus SIMT

SIMT = Single Instruction Multiple ***Threads*** (NVIDIA term)

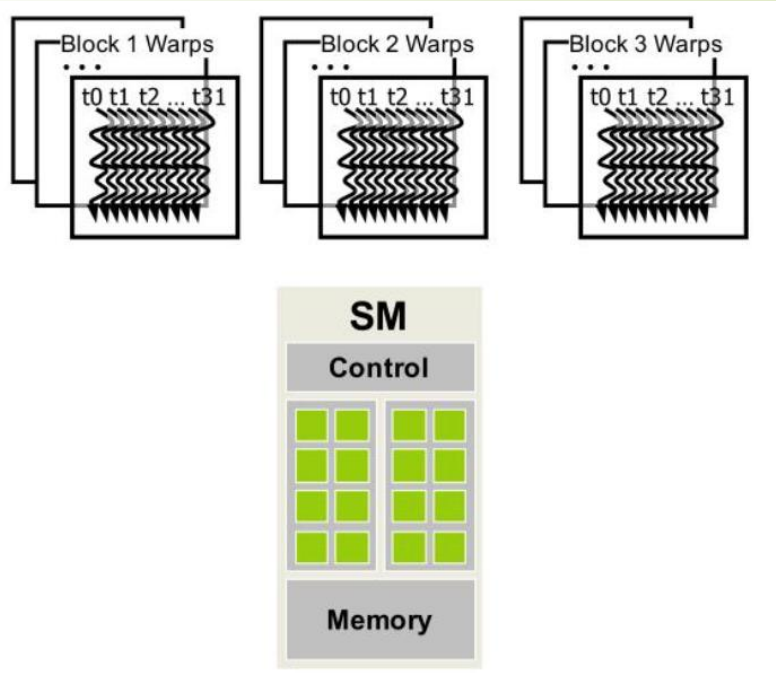
Both models run one instruction over many ALU lanes at once.
They differ in **what the programmer writes** and **what the hardware automatically**

CPU SIMD
the *vector* is the unit of programming:
one thread issues instructions on wide registers,
programmer (or compiler) explicitly manage width, masking, and data layout.

NVIDIA's SIMT
the *scalar thread* is the unit of programming:
programmer declares independence of cuda kernel threads,
the hardware groups 32 threads into a **warp** that runs on SIMD lanes. The hardware implicitly handles masking, divergence, per-lane addressing, and latency hiding at run time.

Warps as Scheduling Units

- Each Block is executed as **32-thread Warps**
  - Once a block is assigned to an SM, it is divided into 32-thread warps.
  - *An implementation decision, not part of the CUDA programming model*



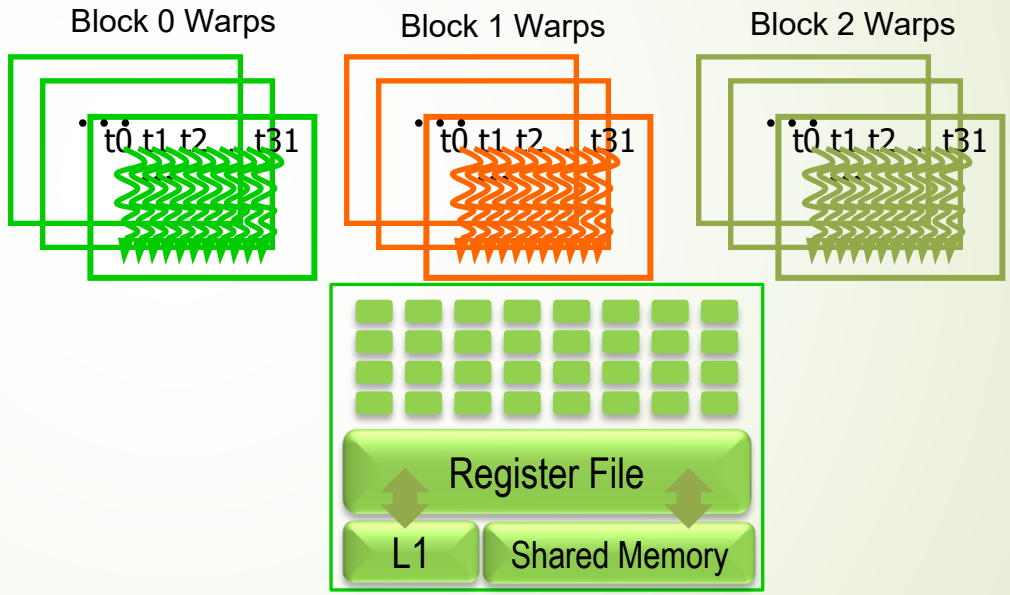
  - Warps are scheduling units in SM
  - Blocks are partitioned into warps based on consecutive increasing thread indices
    - For 1D grid
      - warp0 = threadIdx 0 – 31
      - warp1 = threadIdx 32 – 63
    - For 2D/3D grid
      - dims projected row-major order
  - Threads in a warp execute in SIMD “lanes”
  - Partial warps padded with inactive threads
  - Future GPUs may have different number of threads in each warp

Warp Examples

- If 3 blocks are assigned to an SM and each block has 256 threads, how many Warps are there in an SM?
  - Each Block is divided into 256/32 = 8 Warps
  - There are 8 * 3 = 24 Warps

blockIdx: 0…2

threadIdx: 0…31



- What will the threadIdx-to-warp mapping be for a 3D ZxYxX = 2x8x4 threadblock? Using T(z,y,x)
  - warp0 : thread indices T(0,0,0) … T(0,7,3)
  - warp1 : thread indices T(1,0,0) … T(1,7,3)

Example: Warp/Thread Scheduling (Cont.)

- SM implements **zero-overhead warp scheduling**
  - Warps whose next instruction have operands ready for consumption are eligible for execution
  - Eligible Warps are selected for execution based on a prioritized scheduling policy
  - All threads in a warp execute the same instruction when selected
    - SIMT model (allows writing scalar kernel code which uses SIMD HW)

- Warp-level primitives
  - collection of API functions which provide efficient data exchange and synchronization across threads in a warp
    - `Warp Vote / Match / Reduce / Shuffle / _ _sync`
  - Treat warps as an extra level in software hierarchy (between blocks and threads)
  - Can result in significant performance benefits

Performance Theory and Metrics

Key Concepts

- ***Latency***: time it takes to complete a single task
  - Units: *time*
  - Lower latency is better

- ***Throughput***: rate at which a series of tasks can be completed
  - Units: *work / time*
  - Larger throughput is better
  - Bandwidth (BW): throughput rates with frequency-domain interpretation
    - E.g. memory or communication transaction rate (bytes/sec)
    - *Note: this is not exactly the signal-processing definition* (Hz := s^{-1})
  - Sometimes: *throughput* is actual/achieved rate and BW is HW peak

Bandwidth and Latency

Representative Values (mid-2026)

Notes

- BW: ~10^6 spread from RF to HDD
- Latency: ~10^7 spread

- Bandwidth is buyable (add lanes, channels, wavelengths);
- latency is bounded by physics.

Channel	Bandwidth (peak)	Latency (ns)
Register file (H100, chip aggregate)	~356 TB/s	5.7×10^{-1}
L1 / shared memory (per-SM, chip aggregate)	~10 TB/s	1.7×10^{1}
L2 cache (A100/H100)	~4–7 TB/s	2×10^{2}
HBM3 (H100 SXM)	3.35 TB/s	4.5×10^{2}
HBM3e (H200 / B200)	4.8 TB/s / ~8 TB/s	4.5×10^{2}
DDR5-5600, 2 channels (desktop)	89.6 GB/s	8×10^{1}
8–12 channel DDR5 server	300–600 GB/s	1×10^{2}
NVLink 4 (H100, per GPU)	900 GB/s bidir (450 each way)	$2–3 \times 10^{3}$
PCIe 4.0 ×16	31.5 GB/s each way	1×10^{3}
PCIe 5.0 ×16	63 GB/s each way	1×10^{3}
InfiniBand NDR / 400 GbE	400 Gb/s = 50 GB/s	$1–2 \times 10^{3}$
NVMe Gen4 SSD (sequential)	~7 GB/s	8×10^{4}
NVMe Gen4 SSD (4K random, high QD)	~4 GB/s at 1M IOPS	8×10^{4}
7200 RPM HDD (sequential)	~200 MB/s	8×10^{6}
Gigabit home internet	1 Gb/s = 125 MB/s	$5 \times 10^{6} – 3 \times 10^{7}$
Wi-Fi 6 (80 MHz, 2×2)	1.2 Gb/s PHY, ~700 Mb/s real	$2 \times 10^{6} – 1 \times 10^{7}$

Performance Theory and Metrics

Analysis-Driven Optimization Process

1. Establish baseline reference
2. Define performance target
3. Iteratively modify to improve performance until target reached
4. Start from working version and *validate* results after each modification
   - "bit identical" results may be unrealistic and unnecessary
5. *Measure* performance to evaluate progress

- **Analytical measures** can be used to guide algorithm selection.
  - Give early insights into
    - Fundamental scaling limitations of a parallel algorithm
    - Root causes of poor performance
- **Empirical measures** useful *after implementation* to understand and deal with real world effects ignored by analytical models

Performance Theory and Metrics

Speedup: compares *latency* for solving the identical computational problem on one hardware unit ("worker") versus on *P* workers (cores, processors)

$$S_P = \frac{T_1}{T_P}$$

T_1 is the latency of program with one worker

T_P is the latency of program with *P* workers

Efficiency: ratio of speedup to the number of workers

$$\text{Efficiency} = \frac{S_P}{P} = \frac{T_1}{PT_P}$$

Linear speedup: algorithm runs P times faster with P processors (efficiency = 100%)

Superlinear speedup: efficiency > 100%

Relative speedup: uses serialization of the parallel algorithm as baseline for T_1

Absolute speedup: uses a better serial algorithm for baseline that may not parallelize well

- ★ Both algorithms **must** solve the **identical** computational problem

Scalability

- **Strong scaling**
  - Considers speedup as *P* varies and the problem size remains fixed.
    - Amdahl's Law
- **Weak scaling**
  - Assumes the problem size grows proportionally with *P*.
    - Gustafson-Barsis' Law

Example

Assume problem size is *M* (the working set in main memory).

There are *P* processors. What is the memory per processor, for

Strong scaling?

Weak scaling?

The Growth of Functions

- Computational Complexity
  - USE: estimate growth of a function without worrying about (machine-dependent) constant multipliers and smaller order terms
    - Independent of hardware and software used to implement the algorithm!
  - *Time complexity*: time required to solve problem of particular size
  - *Space complexity*: memory required to solve problem of particular size
- Big-Omicron (big-O) Notation (*upper bound*)

$$O(g(n)) = \left\{ \begin{array}{c} f(n): \text{there exist positive constants } c, n_0 \text{ such that} \\ 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0 \end{array} \right\}$$

- Big-Omega (big-Ω) Notation (*lower bound*)

$$\Omega(g(n)) = \left\{ \begin{array}{c} f(n): \text{there exist positive constants } c, n_0 \text{ such that} \\ 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0 \end{array} \right\}$$

- Big-Theta (big-Θ) Notation (*bounded*)

$$\Theta(g(n)) = \left\{ \begin{array}{c} f(n): \text{there exist positive constants } c_1, c_2, n_0 \text{ such that} \\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0 \end{array} \right\}$$

Growth of Functions

EXAMPLE Show that $f(x) = 3x^2 + 8x \log x$ is $\Theta(x^2)$

$$0 \leq 8x \log x \leq 8x^2 \text{ for } x > 1$$

Thus it follows

$$3x^2 + 8x \log x \leq 11x^2 \text{ for } x > 1$$

So

$$3x^2 + 8x \log x \text{ is } O(x^2) \text{ for } c_2 = 11 \text{ and } n_0 = 1$$

Also

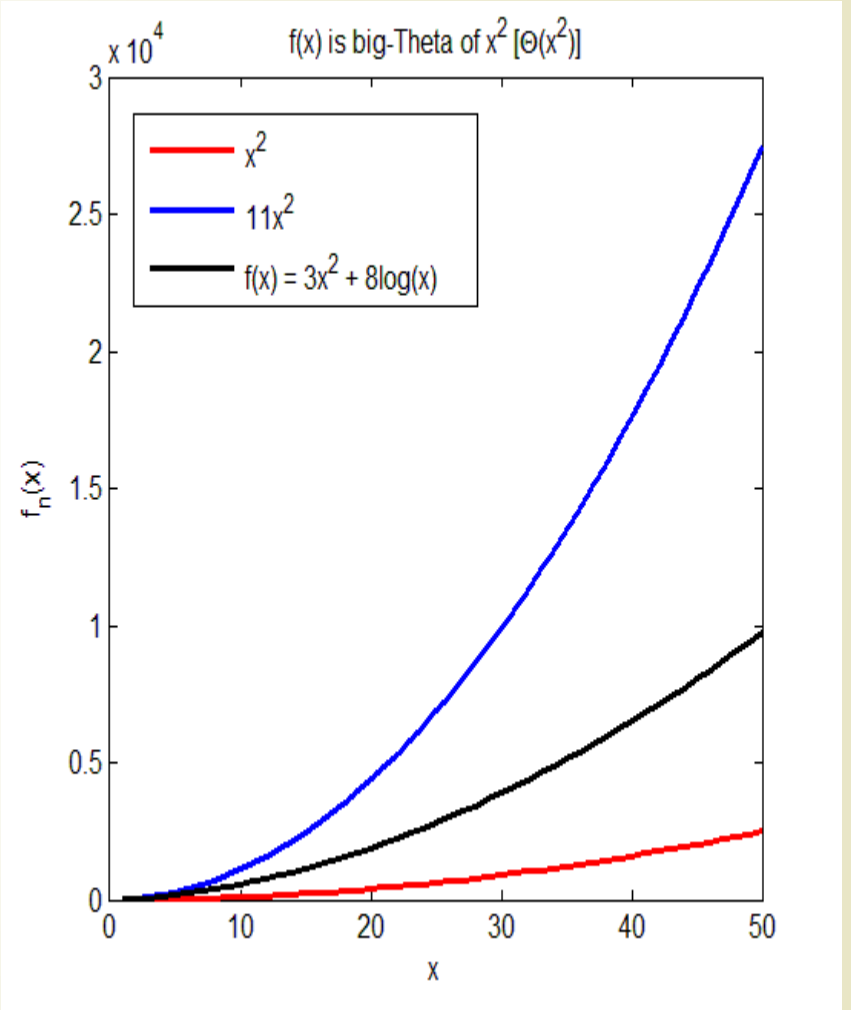
$$0 \leq x^2 \leq 3x^2 + 8x \log x \text{ for } x > 1$$

So

$$3x^2 + 8x \log x \text{ is } \Omega(x^2) \text{ for } c_1 = 1 \text{ and } n_0 = 1$$

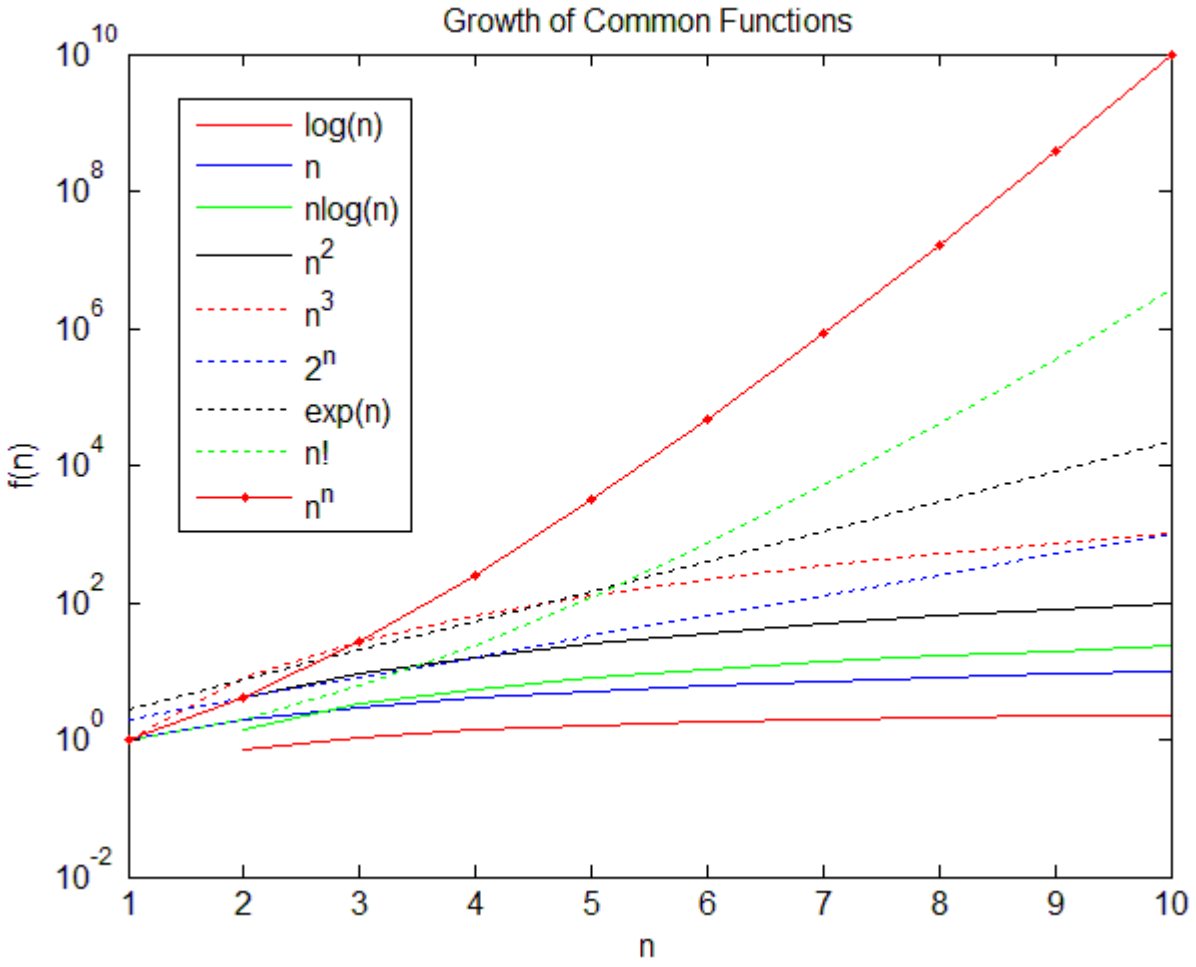
Therefore we have

$$0 \leq c_1 x^2 \leq f(x) \leq c_2 x^2 \text{ for } x > n_0 \text{ with } c_1 = 1, c_2 = 11, n_0 = 1 \text{ THUS } f(x) \text{is } \Theta(x^2) \blacksquare$$

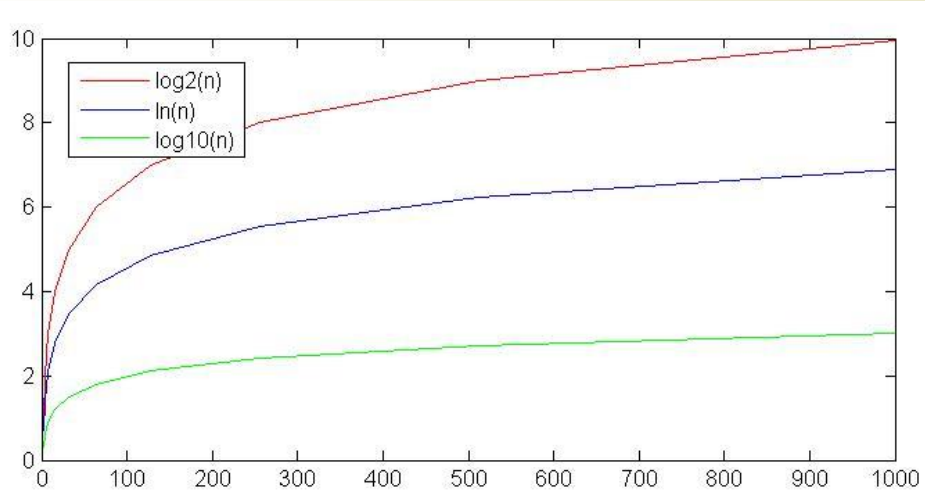


Growth of Functions

- Useful reference plots and terms for common function growth behaviors



Running time	Is said to run in
$\Theta(1)$	constant time
$\Theta(\log n)$	logarithmic 'log' time
$\Theta(n)$	linear time
$\Theta(n^2)$	quadratic time
$O(f(n))$	polynomial time



Asymptotic Speedup and Efficiency

Example: dot-product of two vectors of length N with P workers (assume $P \leq N$)

- Partition vectors between the P workers
- Each computes sub-dot product of length N/P
- Sum sub-products using tree pattern with height $= \log_2 P$

- Asymptotic speedup: $S_P = \frac{T_1}{T_P} \approx \frac{\Theta(N)}{\Theta(N/P + \log_2 P)} = \Theta\left(\frac{N}{N/P + \log_2 P}\right)$

  If $N \gg \log_2 P$ then $AS_P \approx \Theta(P)$

- Asymptotic efficiency: $\mathbf{AE} = \frac{T_1}{P \cdot T_P} \approx \Theta\left(\frac{N}{N + P \log_2 P}\right)$

  If $N \gg P \log_2 P$ or $N \approx P \log_2 P$ then AE $\approx \Theta(1)$

Performance Theory and Metrics

Total execution time T_1 of a program can be divided into

W_{ser} : time spent doing non-parallelizable serial work = **serial fraction**

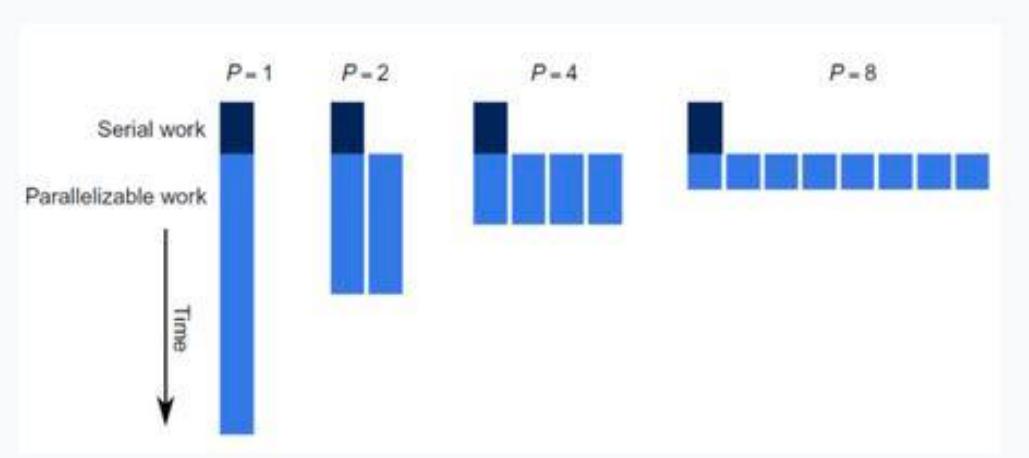
W_{par} : time spent doing parallelizable work

Then given *P* workers

$$T_1 = W_{ser} + W_{par} \qquad \text{and} \qquad T_P \geq W_{ser} + \frac{W_{par}}{P}$$

Amdahl's Law: [Gene Amdahl, 1967]

$$S_P \leq \frac{T_1}{T_P} = \frac{W_{ser}+W_{par}}{W_{ser}+\frac{W_{par}}{P}}$$



Performance Theory and Metrics

Amdahl's Law: Corollary

Let f be the non-parallelizable serial fraction of the total work.

Then we can define the following:

$$W_{ser} = fT_1$$
$$W_{par} = (1-f)T_1$$

And substituting into Amdahl's Law gives $$S_P \leq \frac{1}{f + \frac{1-f}{P}}$$

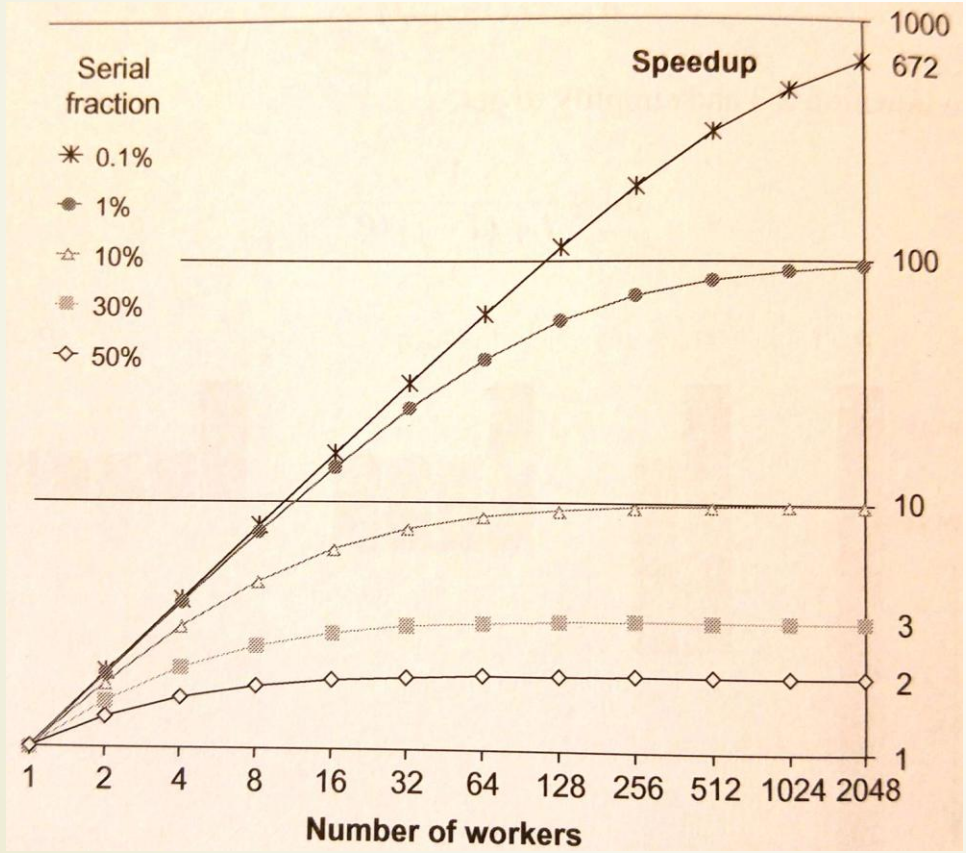
When *P* tends to infinity $$S_\infty \leq \frac{1}{f}$$

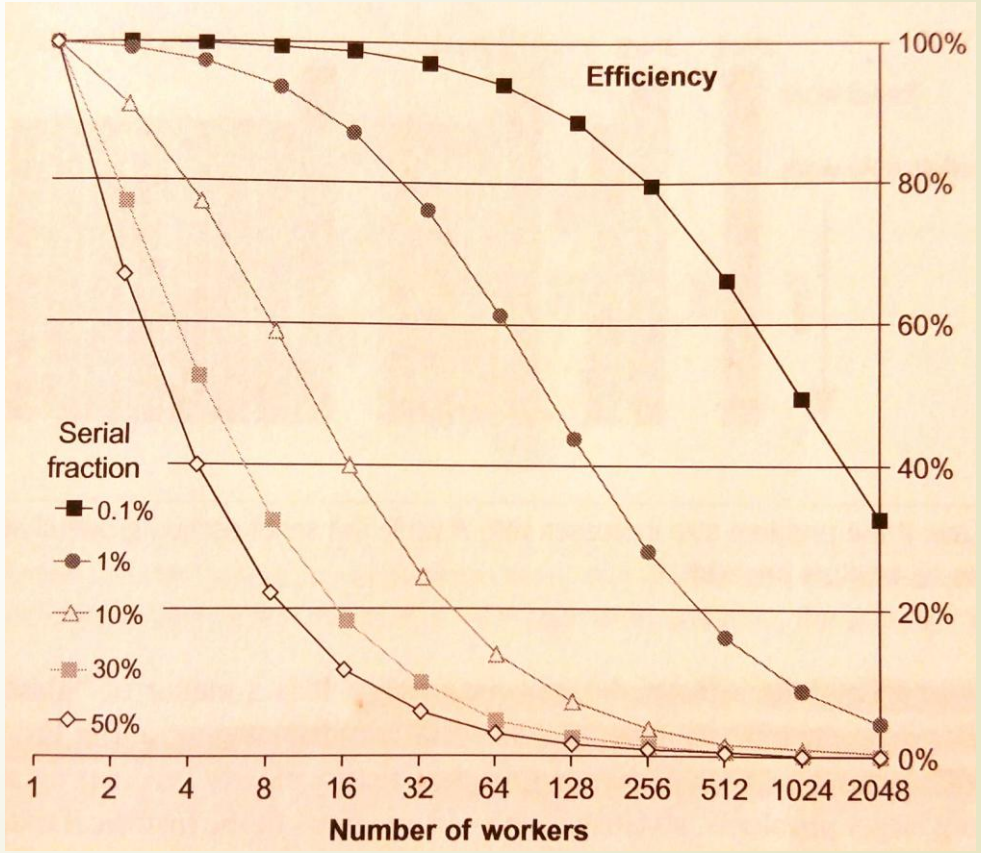
*Speedup is **fundamentally limited** by the fraction of work that is not parallelizable, even with an infinite number of processors.*

Performance Theory and Metrics

Amdahl's Law

Views program size as fixed and computers as changeable





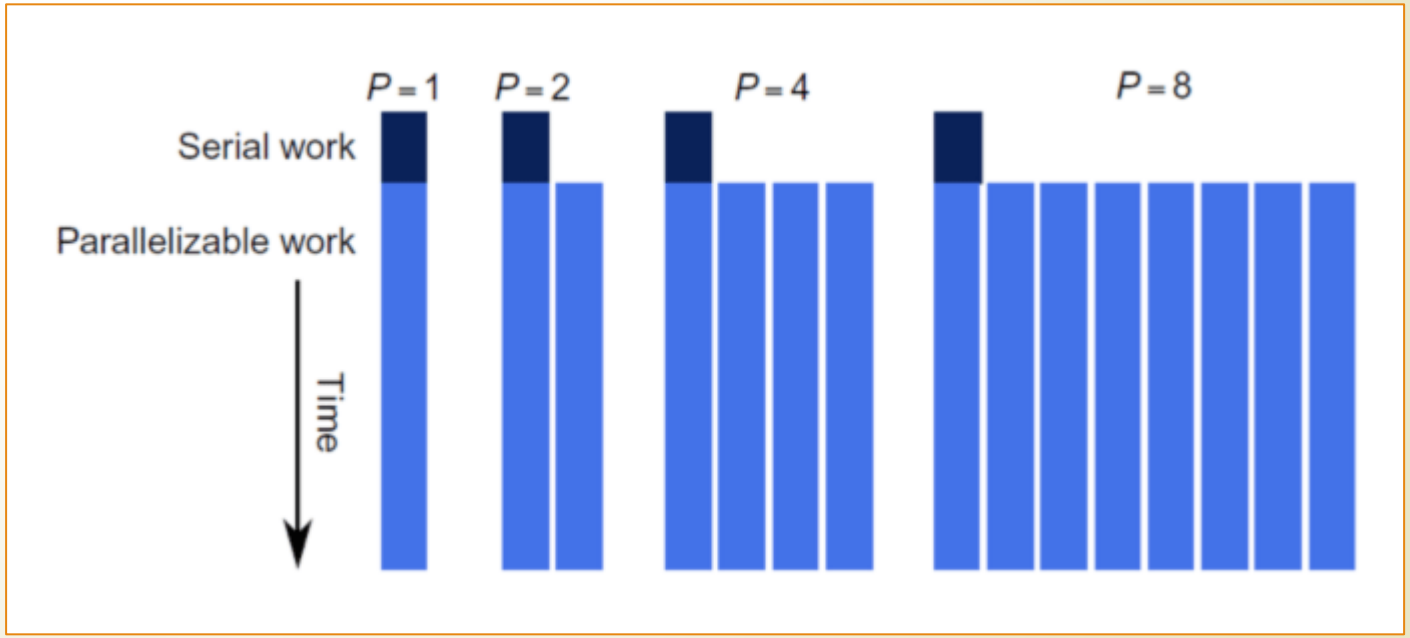
Performance Theory and Metrics

Gustafson-Barsis' Law [John Gustafson, 1988]

Noted that problem sizes grow as computers become more powerful.

- Assume serial portion is constant while parallel portion grows linearly with problem size.
- Serial portion takes same amount of time to execute, but diminishes as fraction of whole.
- Once serial portion becomes insignificant, speedup grows at same rate as number of workers (~linear speedup).

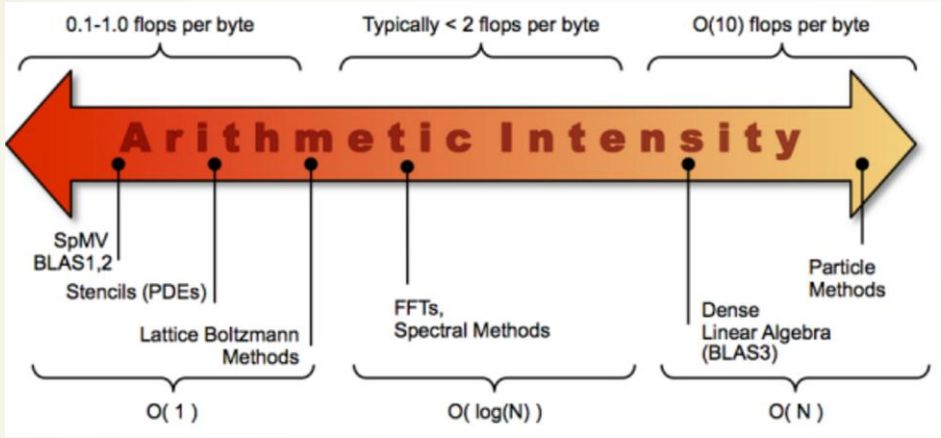
$$S_P \leq \frac{1}{f + \frac{1-f}{P}}$$



Performance Theory and Metrics

Arithmetic Intensity : $AI = \frac{\text{computational work}}{\text{communication}}$

$$AI_{practical} = \frac{FLOPS}{\text{bytes memory traffic}}$$



Example:

Hypothetical device with **100 GB/s** memory bandwidth (i.e. 25 Gfloats/s)

And **625 GFLOP/s** peak single-precision arithmetic throughput

What AI needed for kernel to reach both peak arithmetic and memory throughputs?

AI = 625/100 = 6.25 FLOPs per byte memory transaction

BLAS: Basic Linear Algebra Subroutines

- Categorized into three sets of routines
  - correspond to both chronological order of definition/publication and degree of the polynomial in complexity of algorithms

- Level 1 (1979): operations take *linear* time $O(n)$
  - vector operations on strided arrays: dot products, vector norms
  - generalized vector addition
    - S/DAXPY $\boldsymbol{y} = a\boldsymbol{x} + \boldsymbol{y}$
- Level 2 (1988): operations take *quadratic* time $O(n^2)$
  - matrix-vector operations, including generalized matrix-vector multiplication
    - GEMV: $\boldsymbol{y} = a\mathbf{A}\boldsymbol{x} + b\boldsymbol{y}$
- Level 3 (1990): operations take *cubic* time $O(n^3)$
  - matrix-matrix operations, including generalized matrix multiplication
    - GEMM: $\boldsymbol{y} = a\mathbf{AB} + b\mathbf{C}$

Arithmetic Intensity (AI): Examples

Evaluate AI for 3 standard algorithms from BLAS Levels 1-3

case	BLAS function	
1	S/DAXPY	$\boldsymbol{y} = \alpha\boldsymbol{x} + \boldsymbol{y}$
2	S/DGEMV	$\boldsymbol{y} = \alpha\boldsymbol{A}\boldsymbol{x} + \beta\boldsymbol{y}$
3	S/DGEMM	$\boldsymbol{C} = \alpha\boldsymbol{A}\boldsymbol{B} + \beta\boldsymbol{C}$

S: single precision (float, 32bit)
D: double precision (double, 64bit)

1. Find computational work **W** (FLOPs)
   - Work is property of chosen algorithm and is independent of platform
   - In these (above) cases work depends only on input size: W=W(n)
2. Find communication = memory traffic **Q** (bytes)
   1. Unlike W, Q depends heavily on details of platform, such as cache hierarchy design. Initially, we'll make the "*perfect cache*" assumption
   2. $Q(n) = Q_r(n) + Q_w(n)$: read and write data traffic differs

Calculating FLOP/s

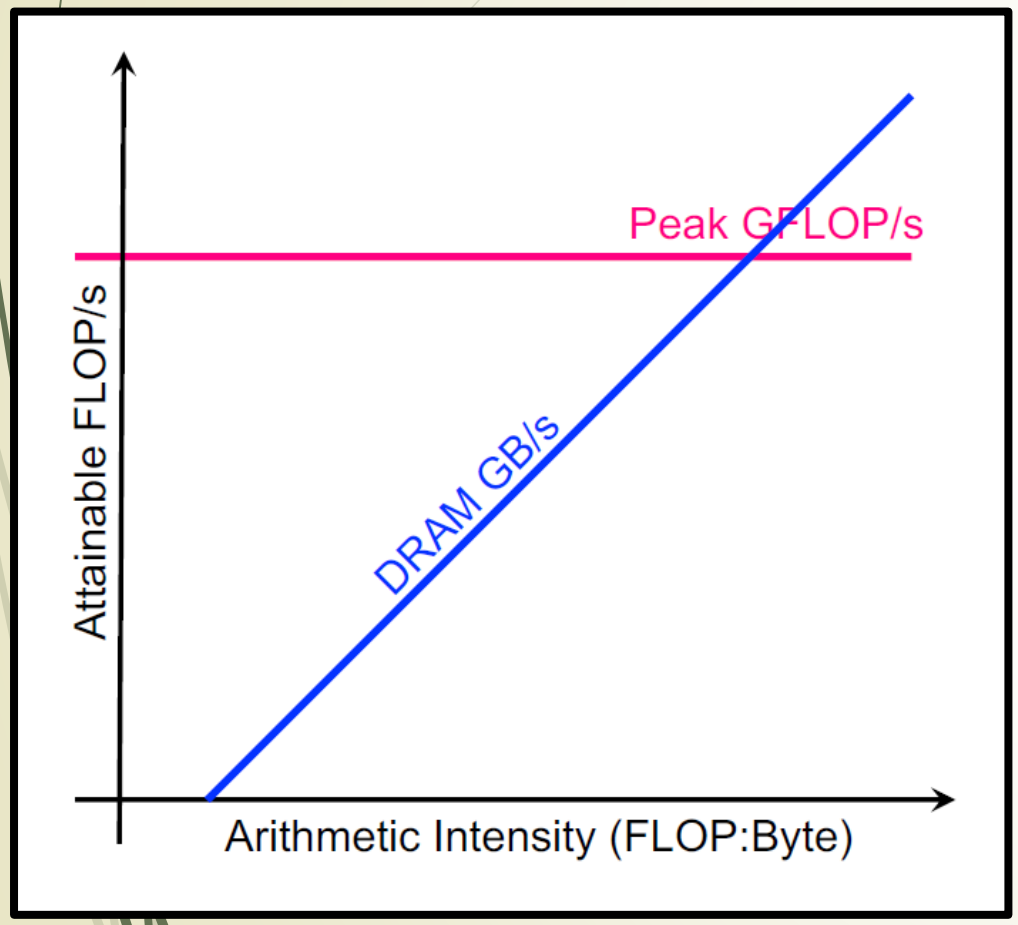
- typically FLOP = one floating-point operation.
  - multiplication, addition, subtraction, comparison of two **single**-precision floating point numbers
  - division, square-root, other complex operations must be considered separately, they are more costly in terms of clock-cycles (*special function unit: SFU*)
  - **double**-precision operations generally take twice as long
- Example: basic Matrix Multiplication

- **MIPS**: Million-of-Instructions-Per-Second: measure of integer performance
- **TOPS**: Tera-Operations-Per-Second: term has drifted from "integer ops" to "low-precision tensor ops of any kind."

Arithmetic Intensity: Examples

case	BLAS function	
1	DAXPY	$\boldsymbol{y} = \alpha\boldsymbol{x} + \boldsymbol{y}$
2	DGEMV	$\boldsymbol{y} = \alpha\boldsymbol{A}\boldsymbol{x} + \beta\boldsymbol{y}$
3	DGEMM	$\boldsymbol{C} = \alpha\boldsymbol{A}\boldsymbol{B} + \beta\boldsymbol{C}$

Roofline Performance Model [Williams, 2009]



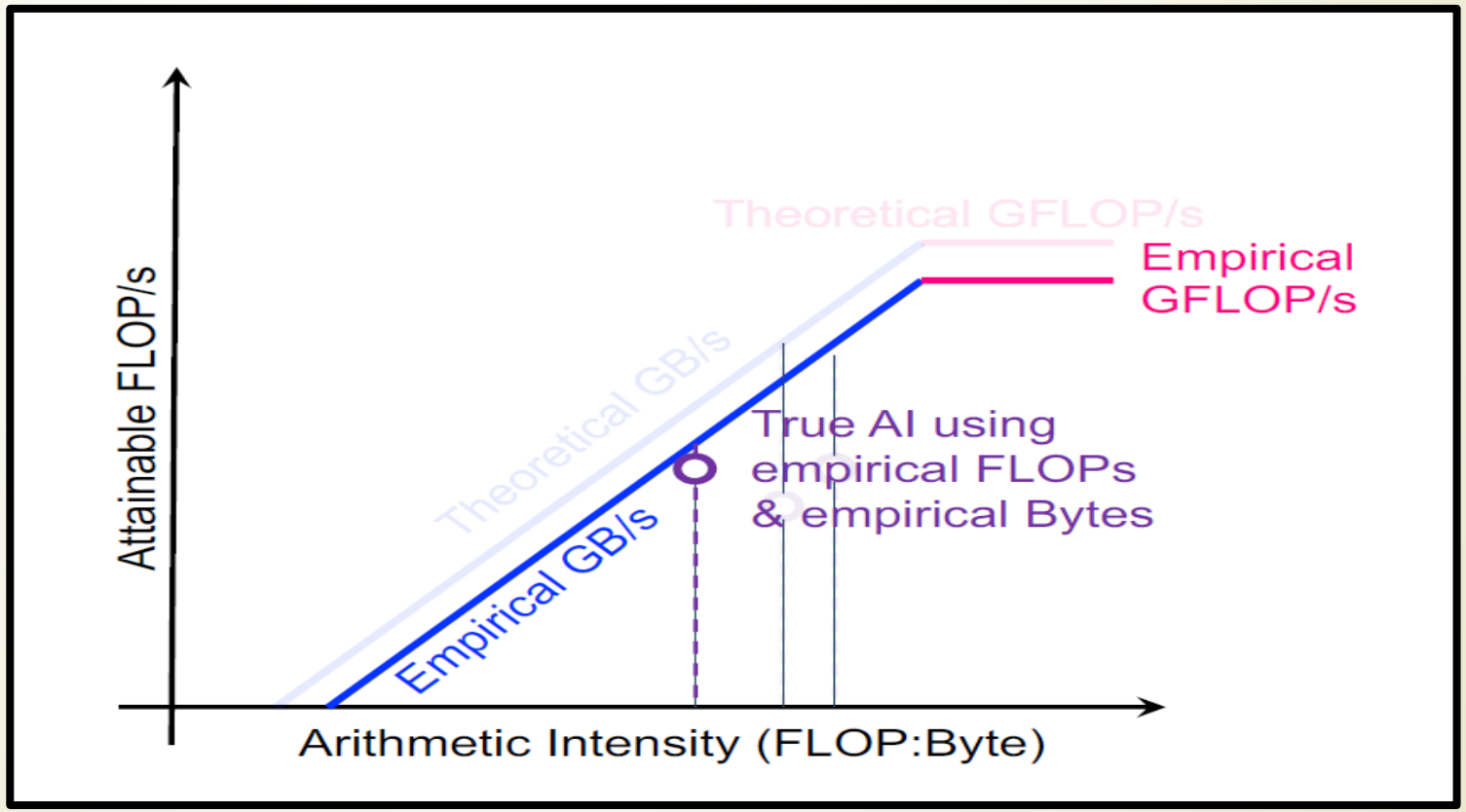
- The Roofline model is oriented around the interplay between application *data locality*, data bandwidth, and computational throughput
- Visually relates performance **P** and arithmetic intensity **AI** of a given kernel to a *platform's peak performance* and *memory bandwidth*
- memory bandwidth becomes additional upper bound for low-AI kernels
- clearly distinguishes *memory-bound* and *compute-bound* kernels

- Performance P : FLOP/s
- Arithmetic Intensity: FLOPS/byte (aka operational intensity)
- Platform's peak performance: FLOP/s
- Platform peak memory bandwidth: bytes/s

Roofline Performance Model

Roofline: theoretical vs empirical

- theoretical performance based on system specs can be highly optimistic
- need empirical (measured) performance data
  - How a specific kernel implementation performs on a specific system



SUMMARY: Performance Theory and Metrics

- Primary purpose of parallelization is **performance**
  - Reduce *total time* to compute a single result (**latency**)
  - Increase *rate* at which a series of results are computed (**throughput**)
  - (Reduce power consumption of a computation)

Important General Considerations for Parallel Algorithm Design

- Total amount of **computational work**
- Total amount of **communication** (memory access)
  - Communication costs are not free, uniform, or linear
  - **Locality**: closeness in space and/or time
- The **span** (critical path) of the code
  - Critical factor for scalability
  - Includes the **serial fraction** of the code



And now, with no further ado…

MATRIX MULTIPLICATION on GPU

Matrix Multiplication Powers Modern AI

Llama 3.1 405B was trained on 3.8×10^{25} FLOPs — and nearly every one of them was a GEMM

~99%

of a dense Transformer's training FLOPs are matrix multiplies: the linear layers plus attention QK^T and PV

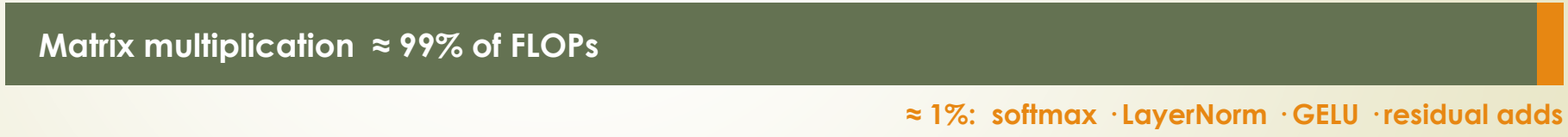
6ND

training FLOPs ≈ 6 × parameters × tokens — 2 per weight in the forward GEMM, 4 in backward

15×

H100 Tensor Core BF16 GEMM throughput vs. its FP32 CUDA cores: 989 vs. 67 TFLOPS

Where a Transformer's FLOPs go — and why the other 1% still matters



That 1% runs on the non-Tensor-Core path at ~1/15 the throughput — left unfused it can dominate runtime, which is why kernels like **FlashAttention** fuse it into the GEMM.

Sources: Kaplan et al. 2020 (6ND); Dao 2023, FlashAttention-2 (matmul vs. non-matmul throughput); NVIDIA H100 SXM datasheet (dense); Meta, Llama 3.1 report (2024)

Matrix Multiplication

C = **AB**

where **A** is M x P, **B** is P x N

and **C** is M x N

notation: matrix element $c_{row,col}$

$$c_{i,j} = \sum_{k=0}^{P-1} a_{ik} b_{kj} = a_{i0} b_{0j} + a_{i1} b_{1j} + \cdots + a_{i(k-1)} b_{(k-1)j}$$

$$\begin{bmatrix} c_{00} & c_{01} & c_{02} & c_{03} \\ c_{10} & c_{11} & c_{12} & c_{13} \\ c_{20} & c_{21} & c_{22} & c_{23} \\ c_{30} & c_{31} & c_{32} & c_{33} \end{bmatrix} \quad \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} \quad \begin{bmatrix} b_{00} & b_{01} & b_{02} & b_{03} \\ b_{10} & b_{11} & b_{12} & b_{13} \\ b_{20} & b_{21} & b_{22} & b_{23} \\ b_{30} & b_{31} & b_{32} & b_{33} \end{bmatrix}$$

Algorithm: Theoretical complexity and AI

- For each output element $c_{i,j}$ we perform dot-product of row-i and col-j
  - this involves P multiplies and P additions
- We do this for MN elements in **C**
- thus MN(2P) operations (FLOPs if doing single-precision floating point math)
- we can say approximately MNP operations
  - if matrices are square M=N=P and we have $O(N^3)$ operations
- $W \sim= N^3$

- What about data movement? (D: data size in bytes, 4=SP, 8=DP)
- Data read: $Q_r = 2*N^2*D$
- Data write: $Q_w = N^2 *D$
- Total: $Q = Q_r + Q_w = 3*N^2*D$

- then $AI = W/Q = N^3 / 3*N^2*D = N / 3D \sim O(N)$

MMULT: Serial Reference

- Generally matrices (h_A, h_B) must be *conforming*
  - inner dimensions are same (P)
- Here we assume matrices have following dimensions
  - h_A : MxP
  - h_B : PxN
  - h_C : MxN

```
// compute CPU serial reference result
   for (int i = 0; i < M; ++i)  // for each row
   {
       for (int j = 0; j < N; ++j) // for each col
       {
           float pval = 0;
           for (int k = 0; k < P; ++k)
           {
               pval += h_A[i * P + k] * h_B[k * N + j];
           }
           h_C[i * N + j] = pval;
       }
   }
```

MMULT: Parallelizing
Naïve - v0 - all global memory (GMEM)

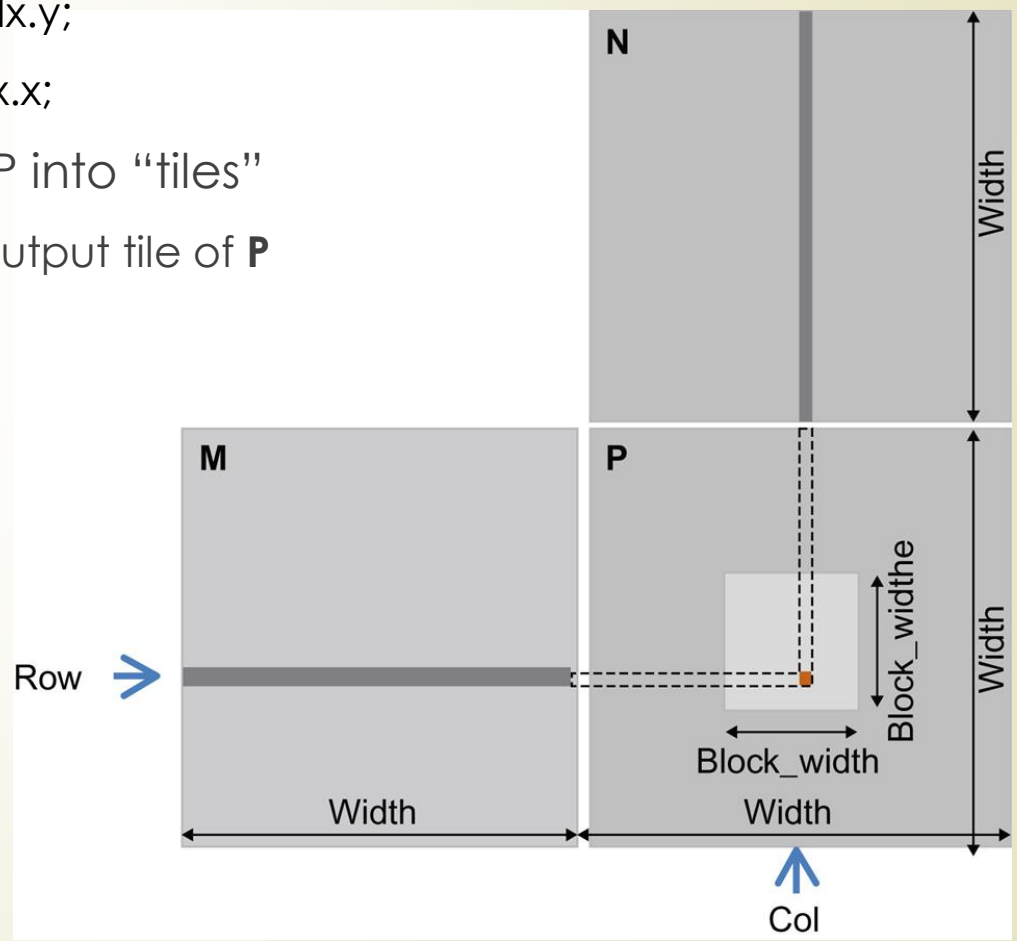
```
// compute CPU serial reference result
   for (int i = 0; i < M; ++i)  // for each row
   {
       for (int j = 0; j < N; ++j) // for each col
       {
           float pval = 0;
           for (int k = 0; k < P; ++k)
           {
               pval += h_A[i * P + k] * h_B[k * N + j];
           }
           h_C[i * N + j] = pval;
       }
   }
```

- GMEM generally should be your first step
  - get the kernel working and computing correct results on GPU
  - provide a reference baseline for future improved variants
- Keep It Super Simple (K.I.S.S.)
  - we'll map the outer two loops to a 2D grid
  - each kernel calculates a single output element
    - and computes the inner loop
- PROS
  - lots of device DRAM = ability to handle larger matrices
  - Conceptually simplest approach
- CONS
  - DRAM is slowest type of device memory
  - need to consider implications of memory access patterns

MMULT: Parallelizing
Naïve - v0 - all global memory

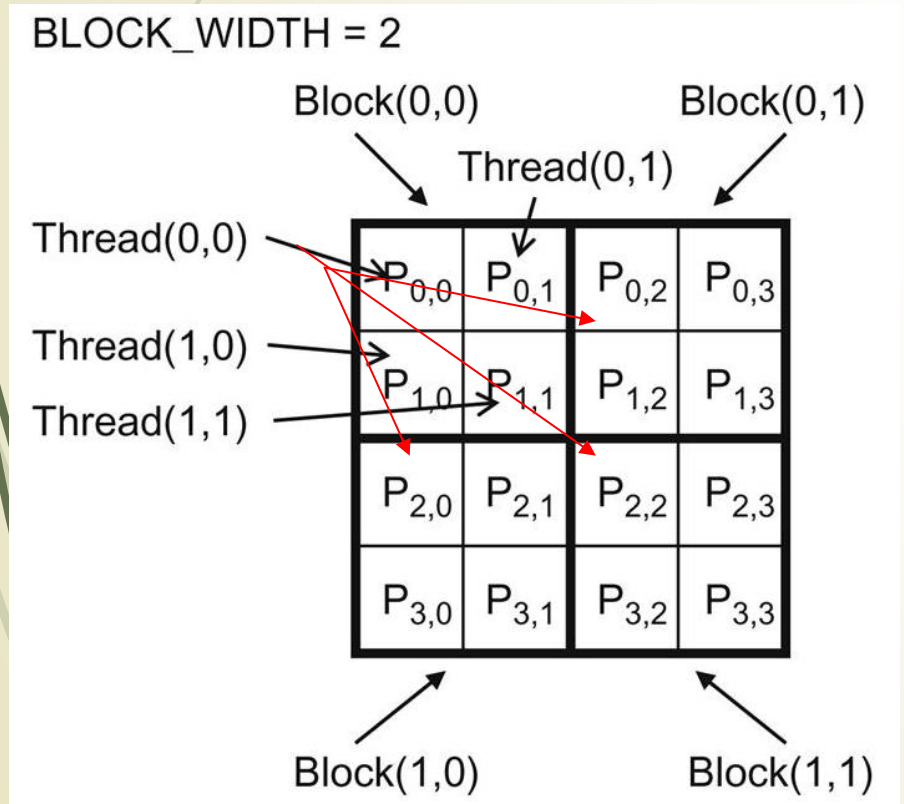
- map kernel threads to elements of output matrix **P**
  - row = blockIdx.y * blockDim.y + threadIdx.y;
  - col = blockIdx.x * blockDim.x + threadIdx.x;
- Threadblock-to-data mapping divides P into "tiles"
  - Each 2D threadblock will calculate an output tile of **P**

```
__global__ void mmult_sqr_gmem(float* M, float* N, float* P, int width)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if ((row < width) && (col < width))
    {
        float pval = 0;
        for (int k = 0; k < width; ++k) {
            pval += M[row * width + k] * N[k * width + col];
        }
        P[row * width + col] = pval;
    }
}
```

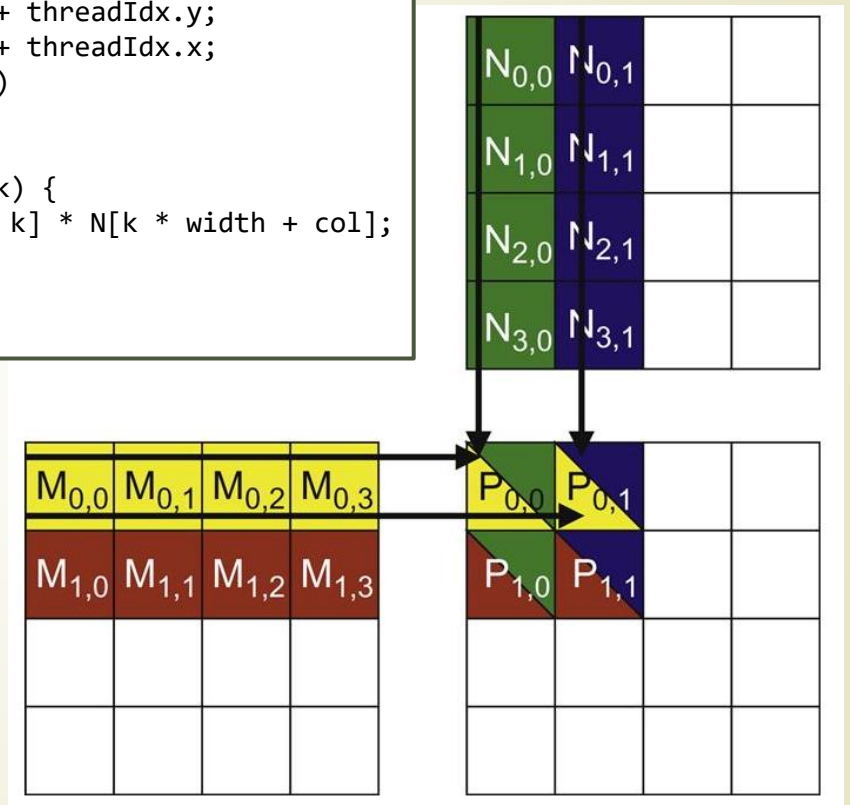


MMULT: v0

- Walk-through a simplified case
  - 2x2 thread blocks, P divided into 4 tiles
- Recall all matrices will be in 1D linearized (row-major) form in device memory




```
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
if ((row < width) && (col < width))
{
      float pval = 0;
      for (int k = 0; k < width; ++k) {
            pval += M[row * width + k] * N[k * width + col];
      }
      P[row * width + col] = pval;
}
```



MMULT: Parallelizing
Naïve - v0 - all global memory

- Effective AI for mmult_sqr_gmem kernel
  - 2*width FLOPs
  - 2*4*width bytes DRAM reads
  - 4 bytes DRAM write
  - As width gets large -> AI = 2*width/8*width ~= ¼ = **0.25 FLOP/B**
    - not great, would help to reduce memory traffic!
- Consider the theoretically achievable AI supported by a specific device
  - AWS Turing T4
    - 40 SMs, 64 FP32 cores/SM = 2560 FP32 cores
    - GPU clock rate = 1.59 GHz (max)
    - Peak Compute: 2560*1.59 * 2 (FMA) = 8.14 TFLOPs
    - DRAM Memory Type: GDDR6 (Double Data Rate = transfers on rising & falling clock edge)
    - Memory clock: 5.001 GHz (max)
    - Memory bus width: 256 bits = 32 bytes
    - Max memory bandwidth = 5.001 * 32 * 2 = 320 GB/sec
    - AI_peak = 8.14 TFLOPS / 320 GB/sec = 25.4 FLOPs/Byte
- So this kernel can only achieve AI/AI_peak = 0.25/25.4 ~ 1% efficiency due to memory traffic

MMULT: Parallelizing
Naïve - v0 - all global memory

- There are further memory considerations that can make matters worse
- Memory access patterns of the threads in a warp
  - We'll study these in more detail over the remaining lectures
  - (*coalesced* global memory access)

Additional HW-imposed limitations (for T4 Turing GPU)

- Max threads / block = 1024 = 32x32 blocks
- Max size of thread block = (1024, 1024, 64)
- Max size of a grid = (2^31, 65535, 65535)
- Thus for 2D grids (due to y-dim limits) the maximum # elements in 1 dim is
  - 32*65535 = 2^5 * 2^16 = 2^21
  - which is 2^42 total elements in a 2D matrix ~ 17 TB (for F32s)
  - Current device DRAM will limit size to (much) less than this.*



CUDA Memory Hierarchy

Device memory architecture and data locality

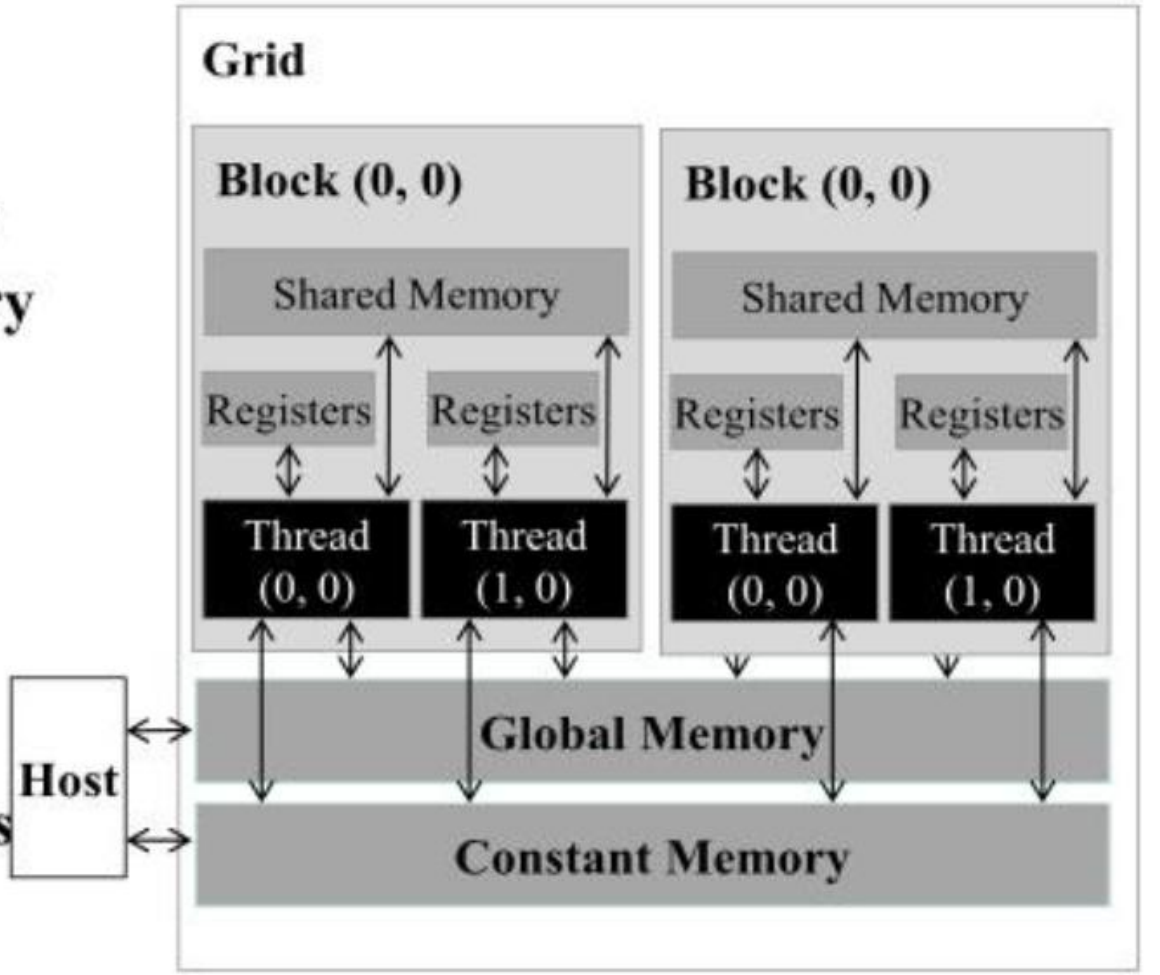
Programmer View of CUDA Memory Spaces

Device code can:

- R/W per-thread **registers**
- R/W per-thread **local memory**
- R/W per-block **shared memory**
- R/W per-grid **global memory**
- Read only per-grid **constant memory**

Host code can

- Transfer data to/from per grid **global** and **constant memories**



Declaring CUDA Variables

Variable declaration	Memory	Scope	Lifetime
int LocalVar;	register	thread	thread
__device__ __shared__ int SharedVar;	shared	block	block
__device__ int GlobalVar;	global	grid	application
__device__ __constant__ int ConstantVar;	constant	grid	application

`__device__` is optional when used with **`__shared__`**, or **`__constant__`**

Automatic variables reside in a register

- Except per-thread arrays that reside in global (local) memory

Shared Memory in CUDA

- A special type of memory whose contents are explicitly defined and used in the kernel source code by the programmer
  - One in each SM
  - On-chip SRAM: Accessed at much higher speed than global memory
    - in both latency and throughput
  - Scope of access and sharing = thread blocks
  - Lifetime = thread block,
    - contents will disappear after the corresponding block finishes and terminates execution
  - Accessed by explicit memory load/store instructions
  - A form of *scratchpad* (programmer controlled) memory in computer architecture

Example:
Shared Memory Variable Declaration Options

Statically-allocated (in kernel)

```
void blurKernel(unsigned char * in, unsigned char * out, int w, int h)
{

  __shared__ float ds_in[TILE_WIDTH][TILE_WIDTH];

…
}
```

Dynamically-allocated (by Host)

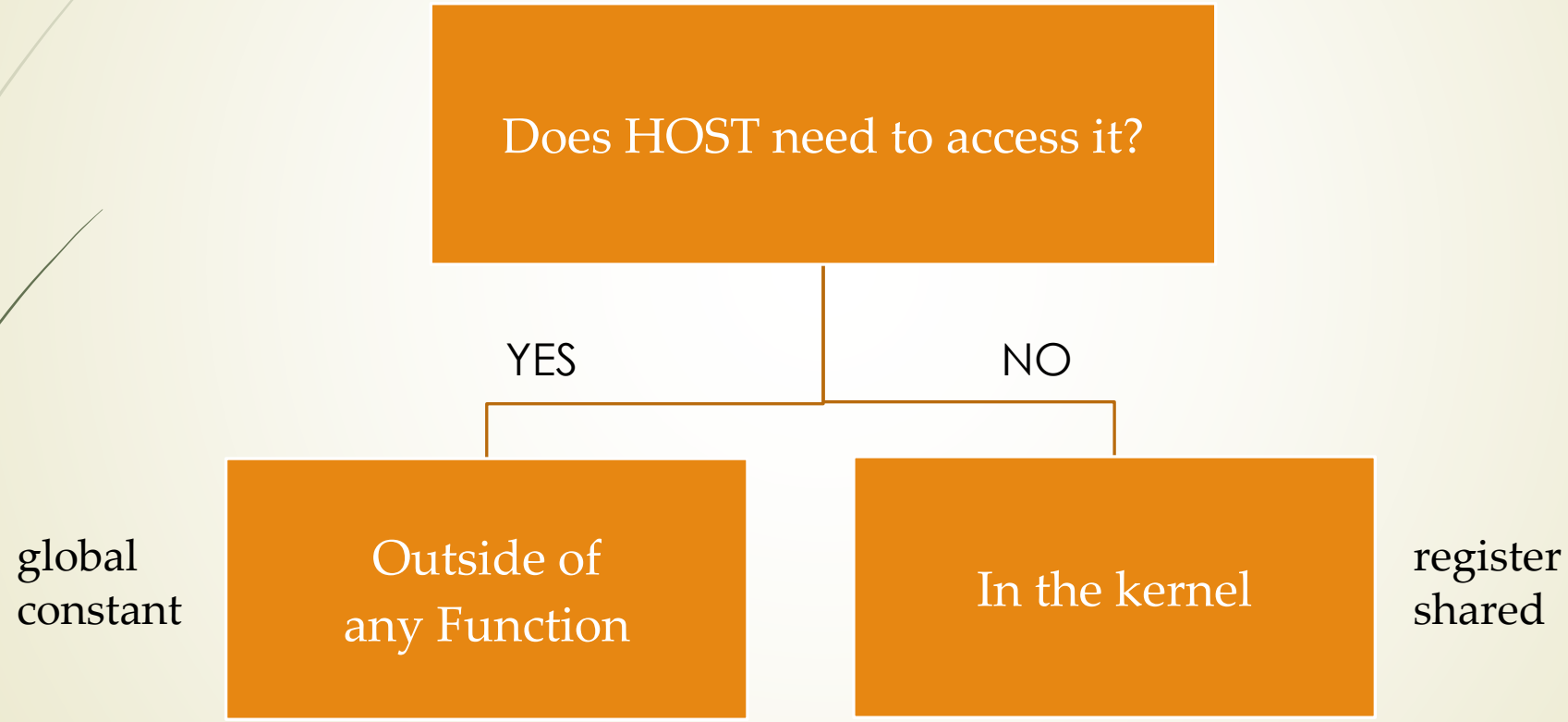
```
// HOST CODE (kernel launch with execution configuration using additional argument)
int shared_mem_size = filterWidth * filterWidth * sizeof(float);
conv_2D_sharedmem<<<blocksPerGrid, threadsPerBlock, shared_mem_size, 0 >>>(d_outimg, d_inimg, d_filt, imgRows, imgCols, filterWidth);
```

```
// KERNEL CODE
extern __shared__ float s_filter[];

__global__ void conv_2D_sharedmem(float* d_output, float* d_input, float* d_filter, int num_row, int num_col,
int filter_size)
{
      // do kernel stuff…

    // threads in block cooperatively copy filter coeffs global -> shared, using block-striding
    for (int i = threadIdx.x + blockDim.x * threadIdx.y; i < filter_size; i += blockDim.x * blockDim.y)
    {
        s_filter[i] = d_filter[i];
    }
```

Where to Declare Variables?



Using appropriate variable declaration specifier keywords!



Improving Parallel MMULT (v1)

shared memory (SMEM) & tiling

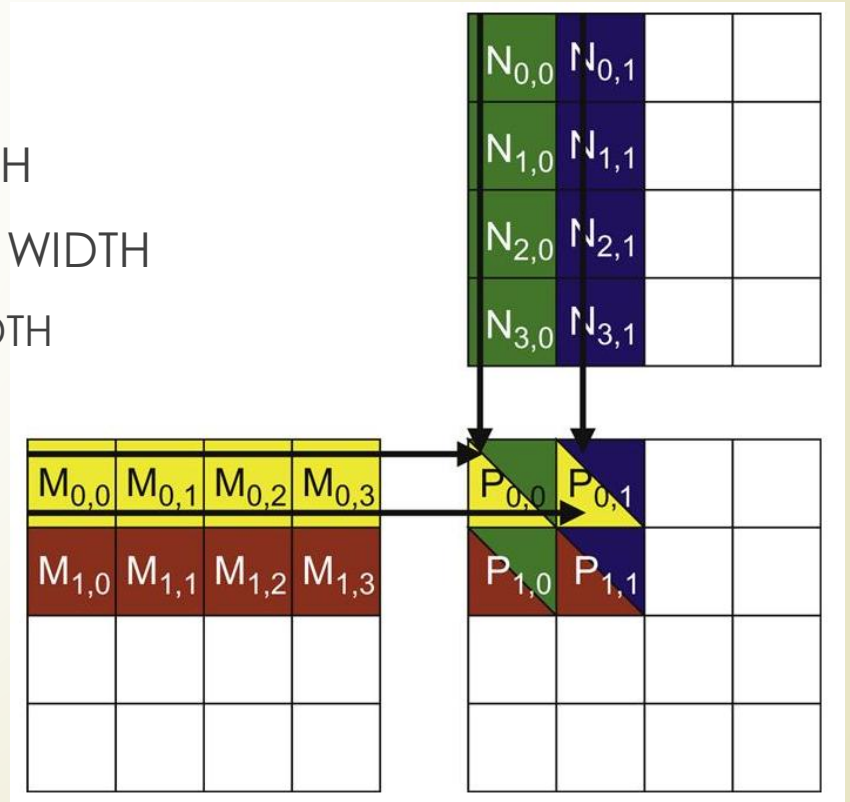
MMULT: Parallelizing
Improved - v1 - shared memory & tiling

- KEY INSIGHT:
  - Each row of M, and each column of N is reused WIDTH times
- Consider the simplified example shown earlier
  - specifically consider thread block (0,0)
    - threads (0,0), (0,1), (1,0), (1,1)
- KEY RESULT: for thread blocks of size WIDTH

  # redundant DRAM accesses = WIDTH
- Potential DRAM traffic reduction to Q/WIDTH

Global Memory Access order (time)

$thread_{0,0}$	$M_{0,0} * N_{0,0}$	$M_{0,1} * N_{1,0}$	$M_{0,2} * N_{2,0}$	$M_{0,3} * N_{3,0}$
$thread_{0,1}$	$M_{0,0} * N_{0,1}$	$M_{0,1} * N_{1,1}$	$M_{0,2} * N_{2,1}$	$M_{0,3} * N_{3,1}$
$thread_{1,0}$	$M_{1,0} * N_{0,0}$	$M_{1,1} * N_{1,0}$	$M_{1,2} * N_{2,0}$	$M_{1,3} * N_{3,0}$
$thread_{1,1}$	$M_{1,0} * N_{0,1}$	$M_{1,1} * N_{1,1}$	$M_{1,2} * N_{2,1}$	$M_{1,3} * N_{3,1}$



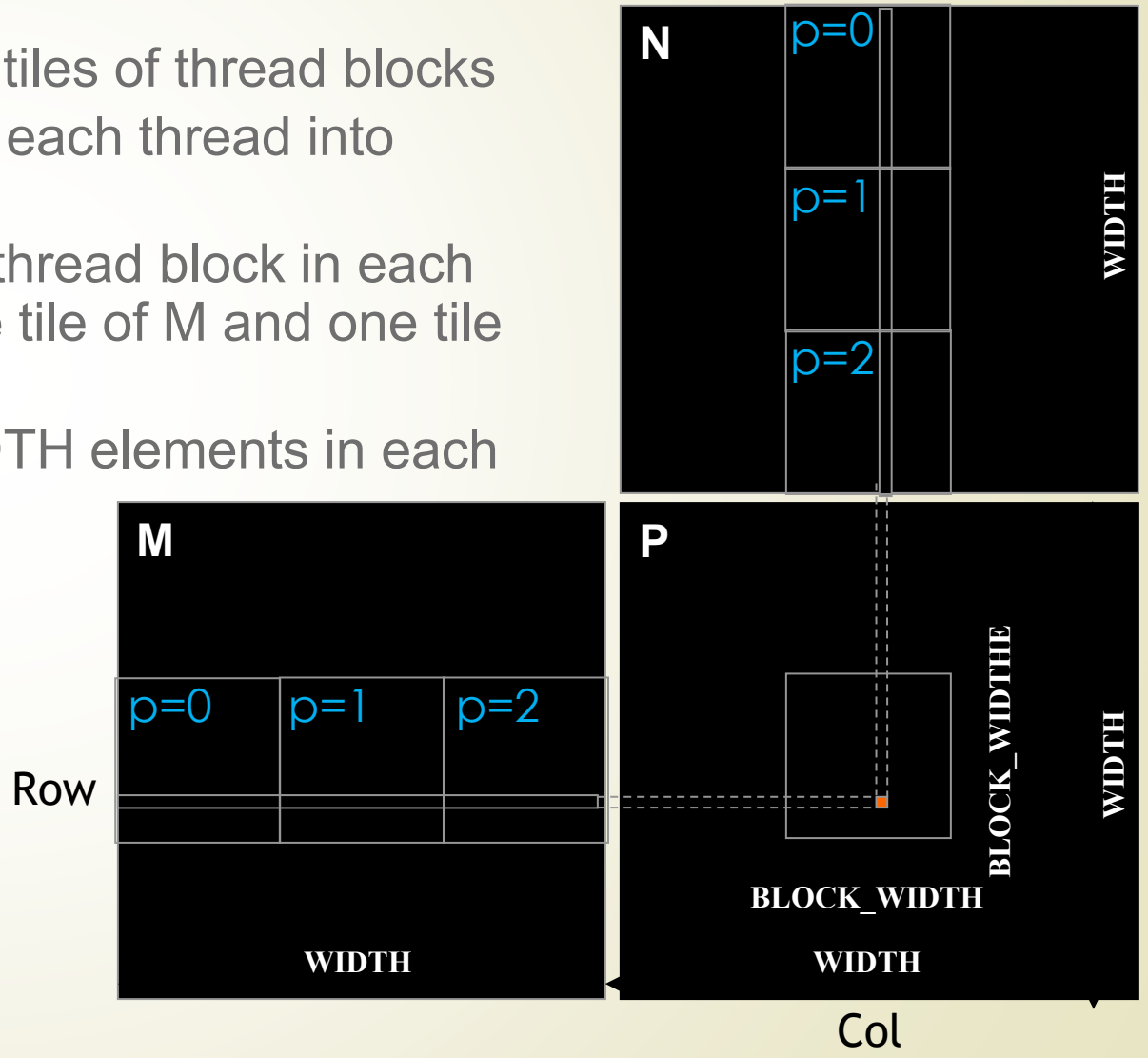
MMULT: Tiled Matrix Multiplication

APPROACH

- Decompose matrices into tiles of thread blocks
- Break up the execution of each thread into **phases** and **tiles**
- the data accesses by the thread block in each phase are focused on one tile of M and one tile of N
- The tile is of BLOCK_WIDTH elements in each dimension

- FOR EACH PHASE
  - Sub-Phases
    1. LOAD (DRAM->SMEM)
    2. USE (compute partial)

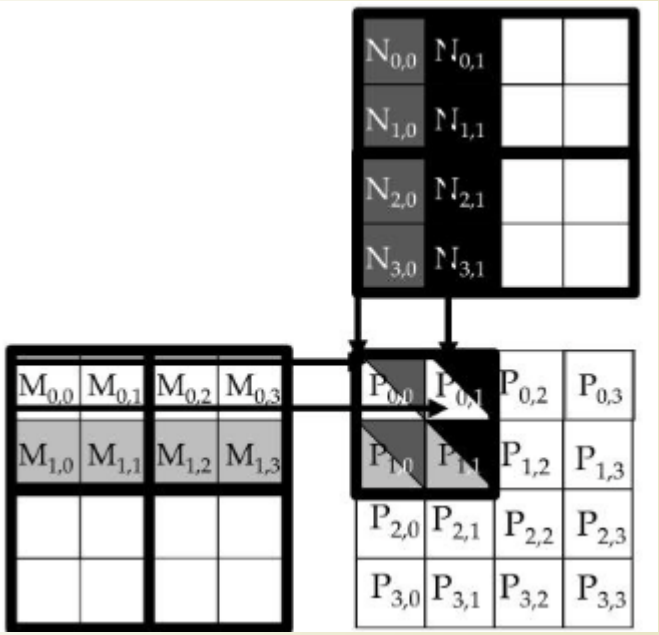
- Synchronization required!



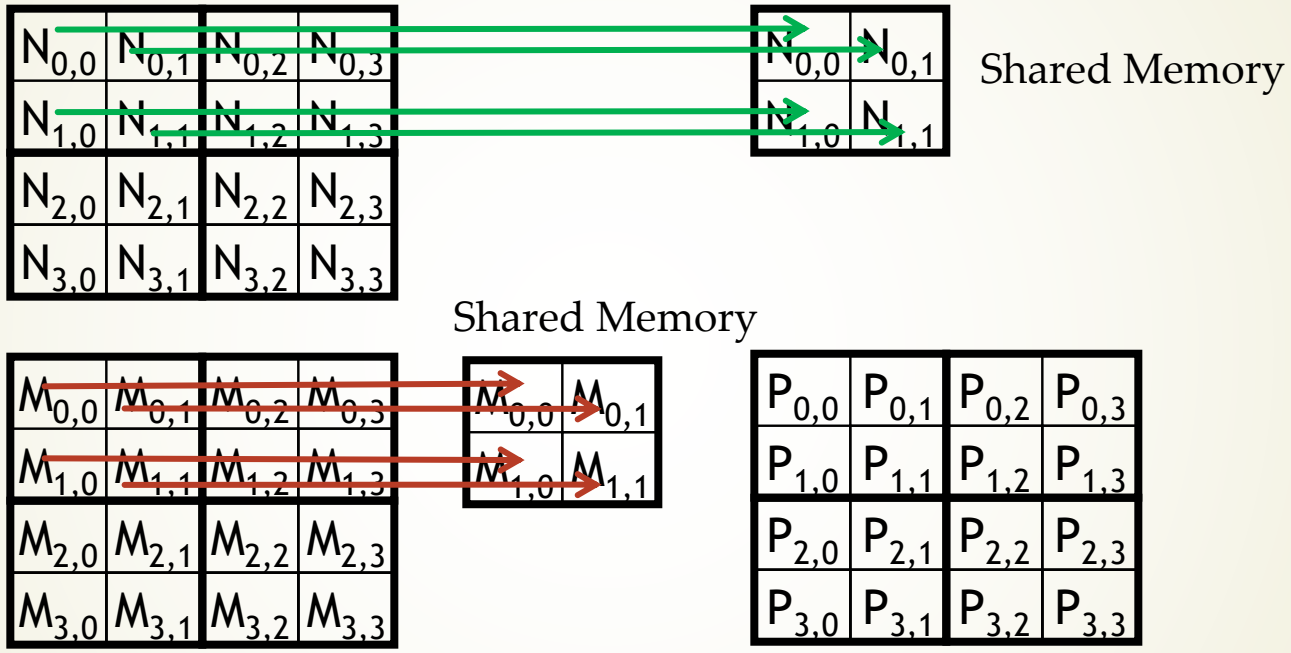
Execution phases of tiled MMULT

	Phase 0			Phase 1		
$thread_{0,0}$	$\mathbf{M_{0,0}}$ ↓ $Mds_{0,0}$	$\mathbf{N_{0,0}}$ ↓ $Nds_{0,0}$	$PValue_{0,0}$ += $Mds_{0,0}*Nds_{0,0}$ + $Mds_{0,1}*Nds_{1,0}$	$\mathbf{M_{0,2}}$ ↓ $Mds_{0,0}$	$\mathbf{N_{2,0}}$ ↓ $Nds_{0,0}$	$PValue_{0,0}$ += $Mds_{0,0}*Nds_{0,0}$ + $Mds_{0,1}*Nds_{1,0}$
$thread_{0,1}$	$\mathbf{M_{0,1}}$ ↓ $Mds_{0,1}$	$\mathbf{N_{0,1}}$ ↓ $Nds_{0,1}$	$PValue_{0,1}$ += $Mds_{0,0}*Nds_{0,1}$ + $Mds_{0,1}*Nds_{1,1}$	$\mathbf{M_{0,3}}$ ↓ $Mds_{0,1}$	$\mathbf{N_{2,1}}$ ↓ $Nds_{0,1}$	$PValue_{0,1}$ += $Mds_{0,0}*Nds_{0,1}$ + $Mds_{0,1}*Nds_{1,1}$
$thread_{1,0}$	$\mathbf{M_{1,0}}$ ↓ $Mds_{1,0}$	$\mathbf{N_{1,0}}$ ↓ $Nds_{1,0}$	$PValue_{1,0}$ += $Mds_{1,0}*Nds_{0,0}$ + $Mds_{1,1}*Nds_{1,0}$	$\mathbf{M_{1,2}}$ ↓ $Mds_{1,0}$	$\mathbf{N_{3,0}}$ ↓ $Nds_{1,0}$	$PValue_{1,0}$ += $Mds_{1,0}*Nds_{0,0}$ + $Mds_{1,1}*Nds_{1,0}$
$thread_{1,1}$	$\mathbf{M_{1,1}}$ ↓ $Mds_{1,1}$	$\mathbf{N_{1,1}}$ ↓ $Nds_{1,1}$	$PValue_{1,1}$ += $Mds_{1,0}*Nds_{0,1}$ + $Mds_{1,1}*Nds_{1,1}$	$\mathbf{M_{1,3}}$ ↓ $Mds_{1,1}$	$\mathbf{N_{3,1}}$ ↓ $Nds_{1,1}$	$PValue_{1,1}$ += $Mds_{1,0}*Nds_{0,1}$ + $Mds_{1,1}*Nds_{1,1}$

time



Phase 0 **Load** for Block (0,0)



- All threads in a block participate
  - for each **Phase** iteration…
  - each thread loads one M element and one N element in tiled code



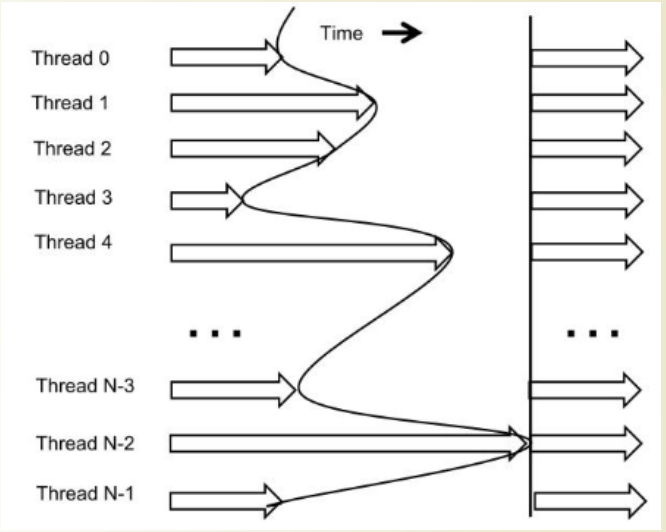
SYNCHRONIZE!

Why?

Barrier Synchronization Usage

- Synchronize all threads in a **thread block**
  - `__syncthreads()` : cuda intrinsic function

THUS CUDA runtime assigns all threads in block to the same SM simultaneously



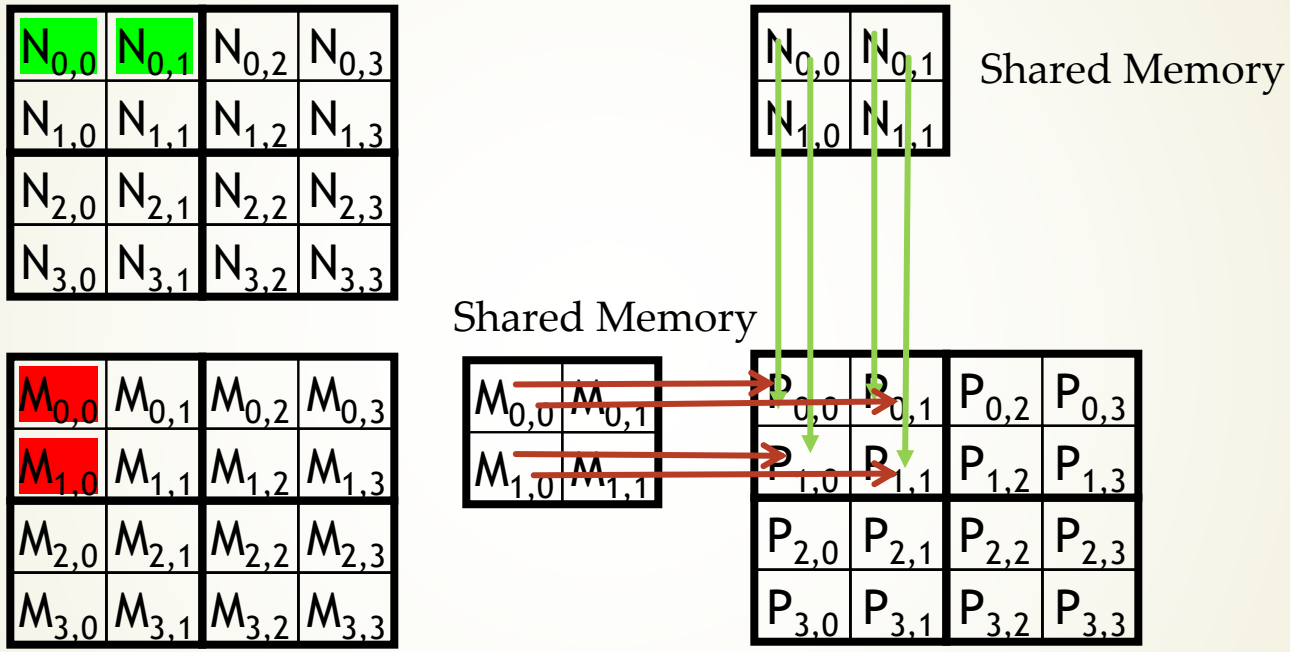
- **All** threads in the same block must reach the __syncthreads() before **any** of them can move on
- All threads must execute
- Otherwise deadlock may result

```
1    void incorrect_barrier_example(int n) {
2         ...
3         if (threadIdx.x % 2 == 0) {
4              ...
5              __syncthreads();
6         }
7         else {
8              ...
9              __syncthreads();
10        }
11   }
```

Barrier Synchronization Usage

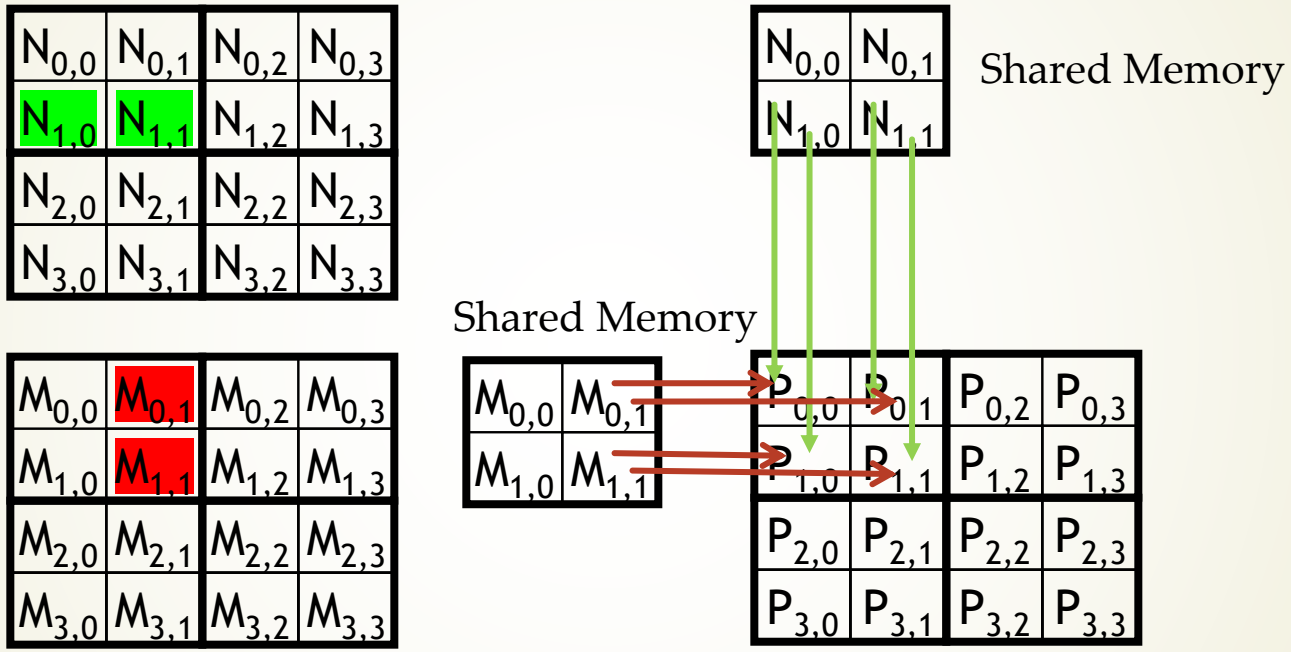
- Used to coordinate the phased execution tiled algorithms
  - To ensure that all elements of a tile are **loaded** at the beginning of a phase
    - All data must be written before other threads read data
    - *Read-After-Write (RAW)* dependency
  - To ensure that all elements of a tile are **consumed** at the end of a phase before proceeding to next load phase
    - Wait for all threads to complete data reads before overwriting
    - *Write-After-Read (WAR)* dependency
- CUDA provides barrier synchronization options at other scopes: warp, grid, cluster
  - We will discuss many of these later in the course
- **ASIDE: intrinsic functions**
  - Special instructions that perform critical functionality or substantial performance improvement
  - Treated in special way by compilers – often translated directly into a corresponding special inline instruction call (not a function call)

Phase 0 **Use** for Block (0,0) (iteration 0)



- All threads in a block participate
  - each thread computes a partial result for the corresponding output **P** element

Phase 0 **Use** for Block (0,0) (iteration 1)



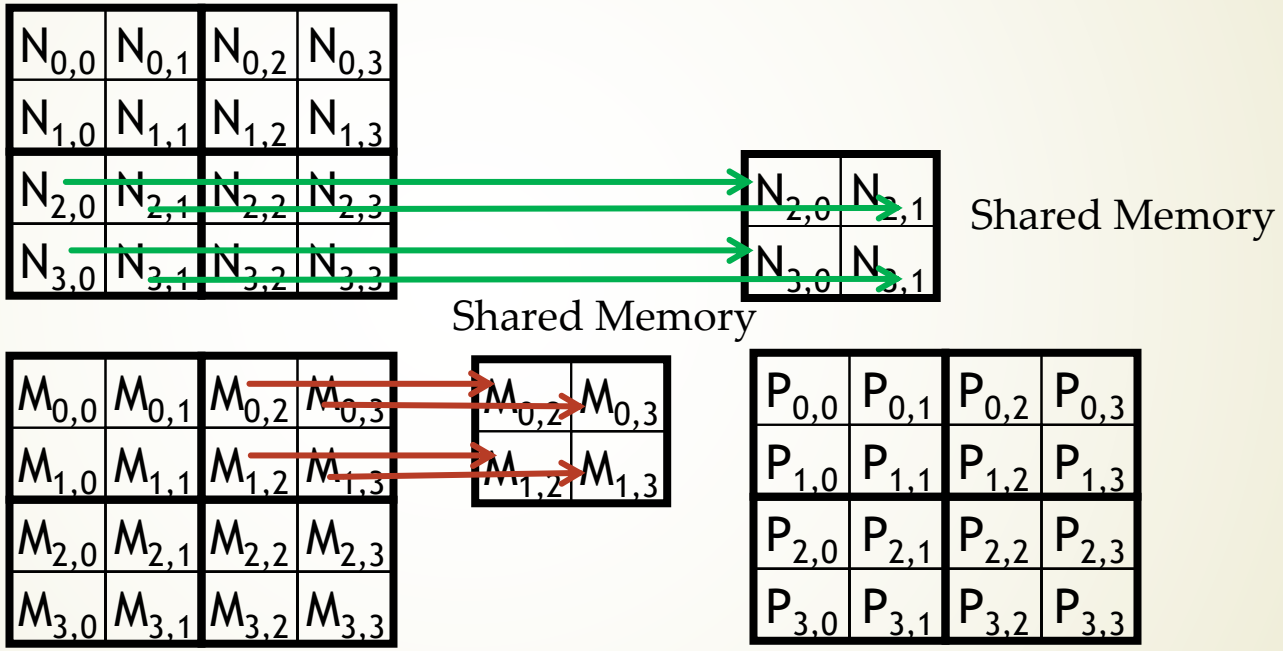


SYNCHRONIZE!

Why?

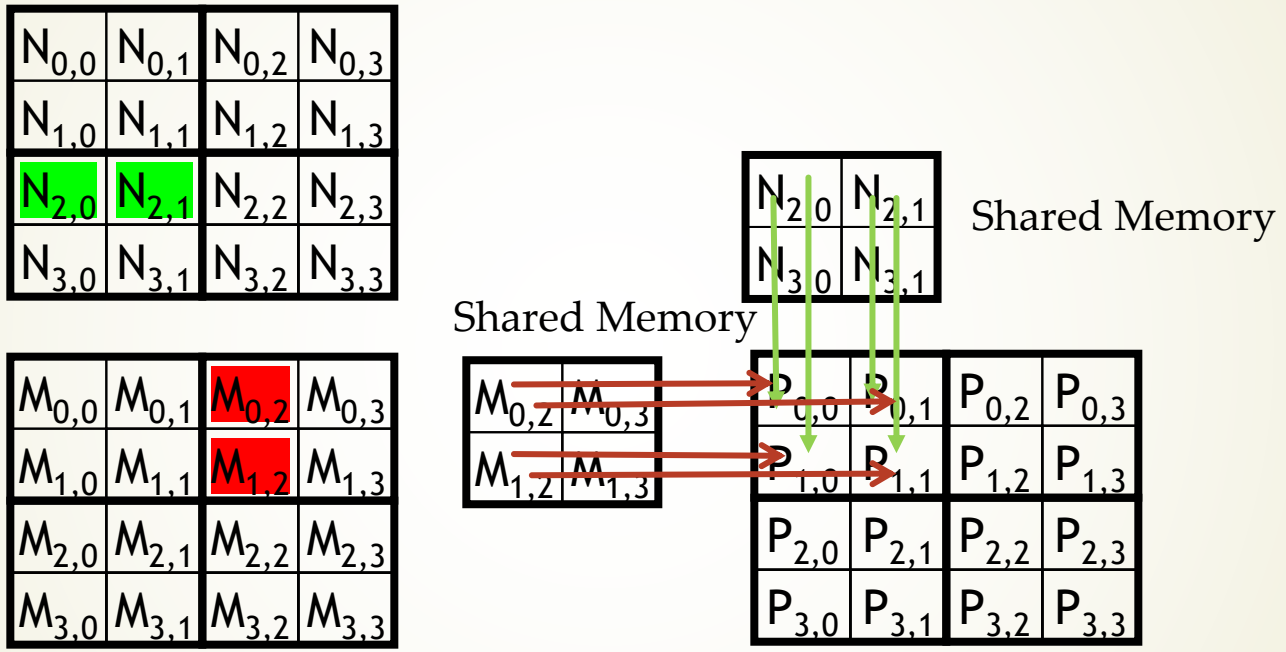
Begin Phase 1

- **Load** for Block (0,0)

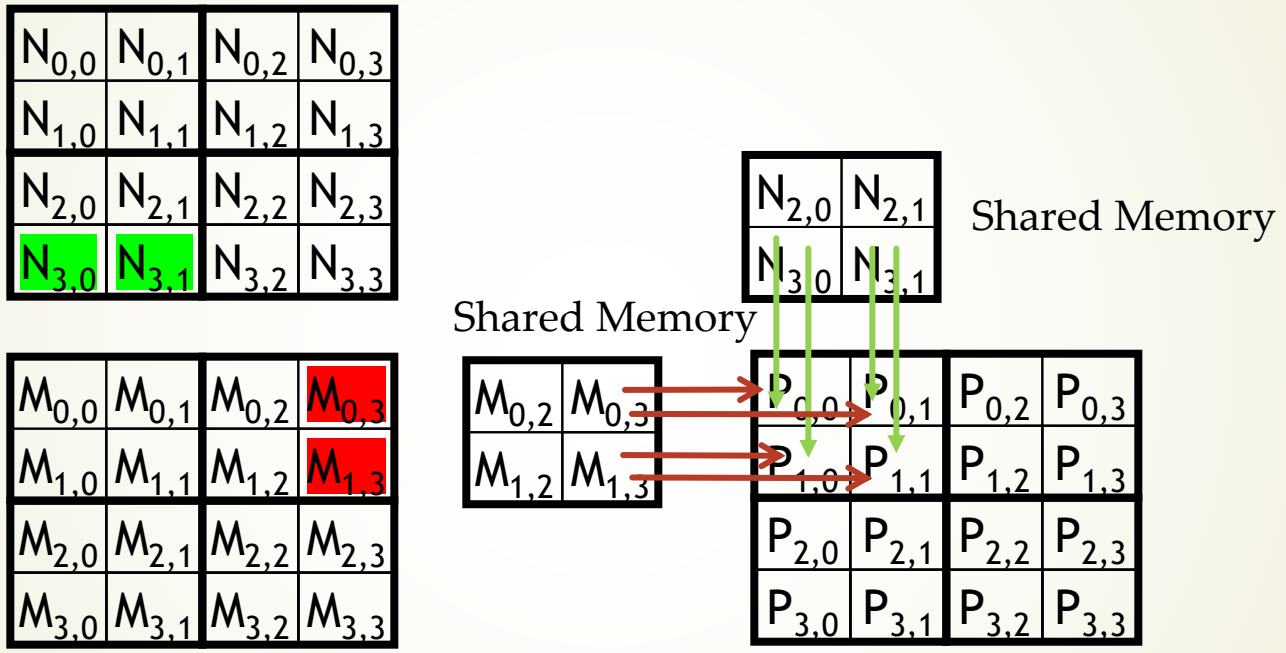


SYNCHRONIZE!

Phase 1 **Use** for Block (0,0) (iteration 0)



Phase 1 **Use** for Block (0,0) (iteration 1)





TILED MMULT – CUDA KERNEL

Phased Tiled Computation with Shared Memory

Implementing the code!

Loading Input Tile 0 of M (Phase 0)

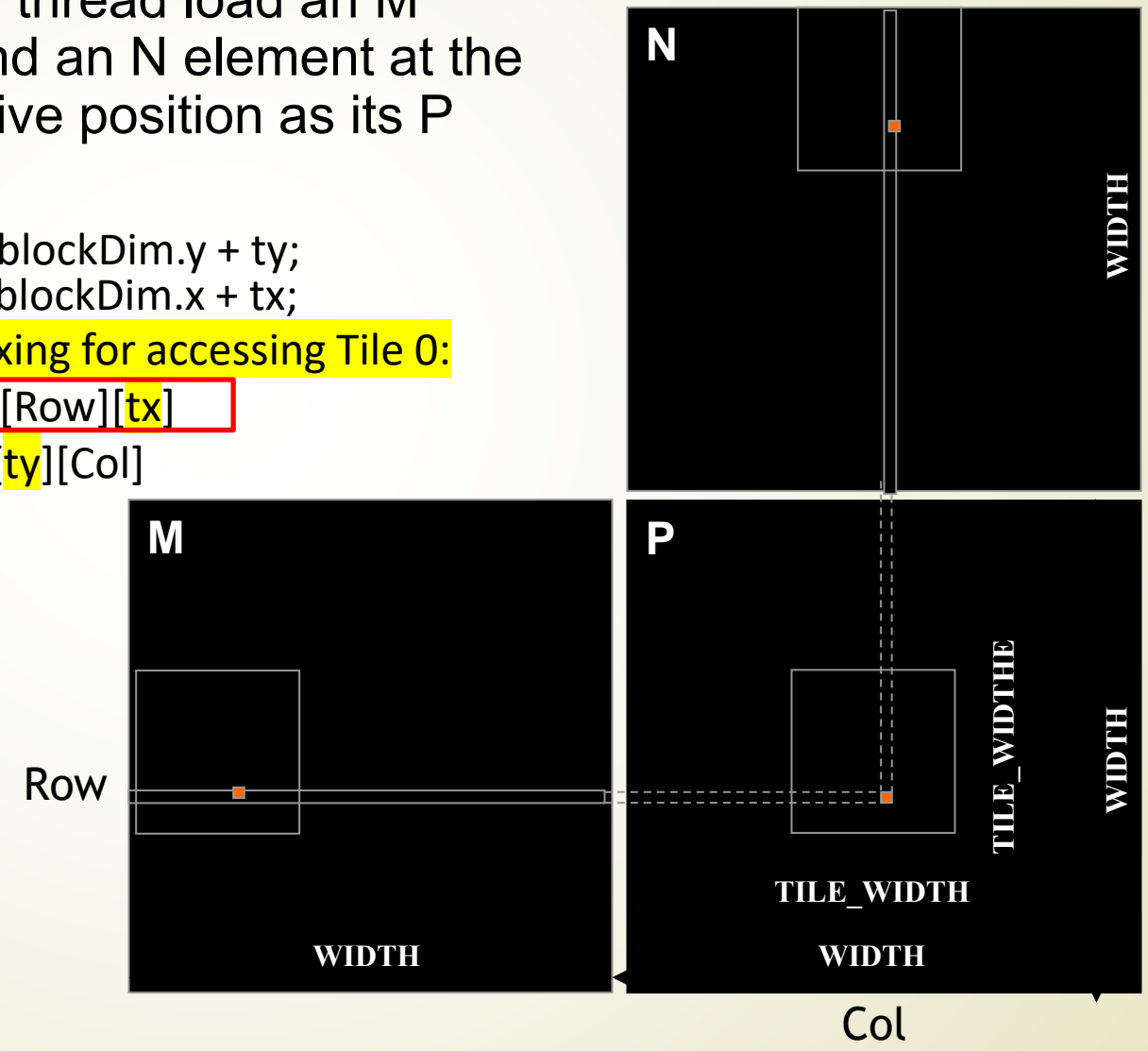
- Have each thread load an M element and an N element at the same relative position as its P element.

```
int Row = by * blockDim.y + ty;
int Col =   bx * blockDim.x + tx;
```

NOTE: 2D indexing for accessing Tile 0:

```
M[Row][tx]
N[ty][Col]
```

phase index p = 0



Loading Input Tile 0 of N (Phase 0)

- Have each thread load an M element and an N element at the same relative position as its P element.

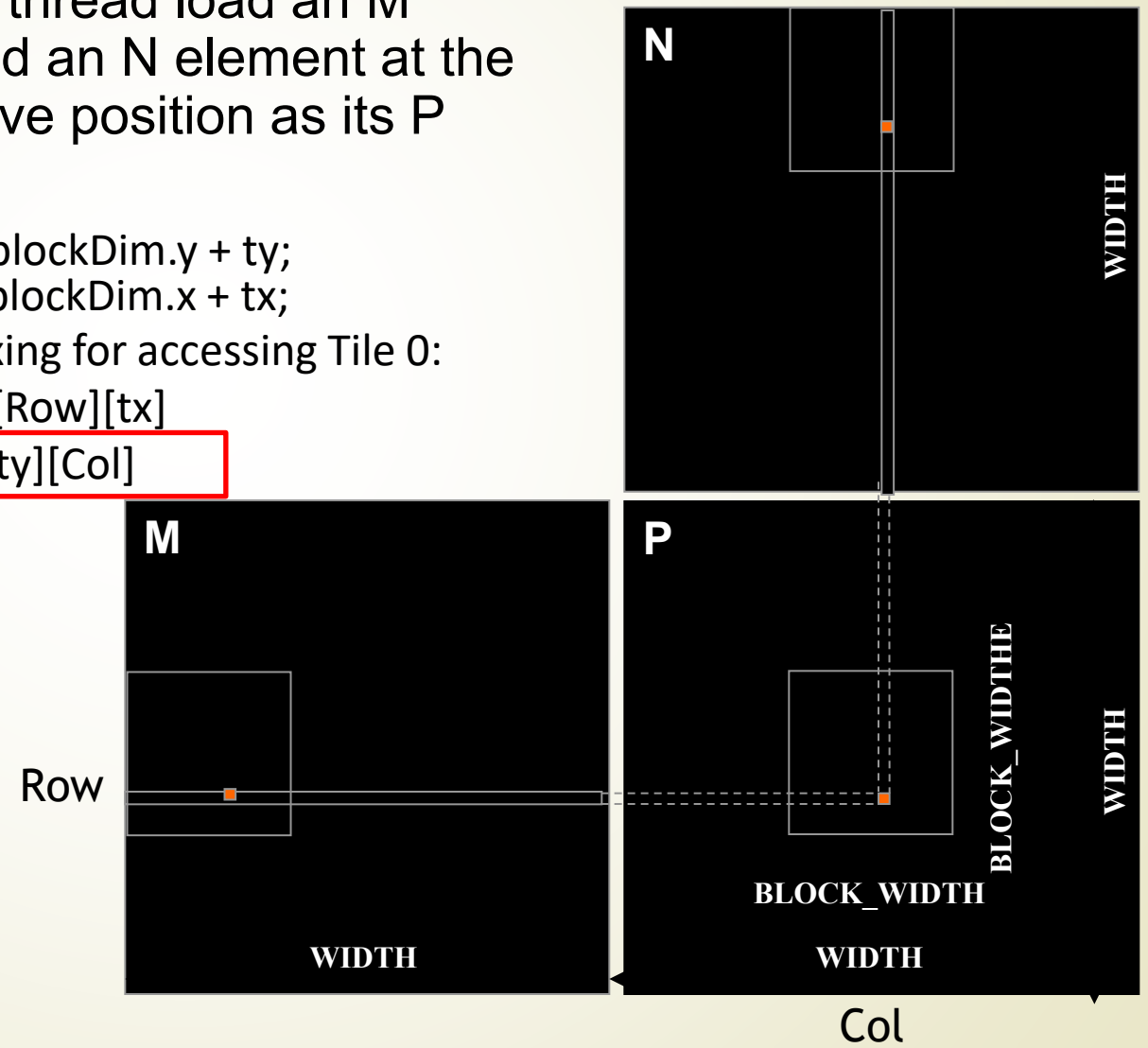
```
int Row = by * blockDim.y + ty;
int Col =   bx * blockDim.x + tx;
```

NOTE: 2D indexing for accessing Tile 0:

M[Row][tx]

N[ty][Col]

phase index p = 0



Loading Input Tile 1 of M (Phase 1)

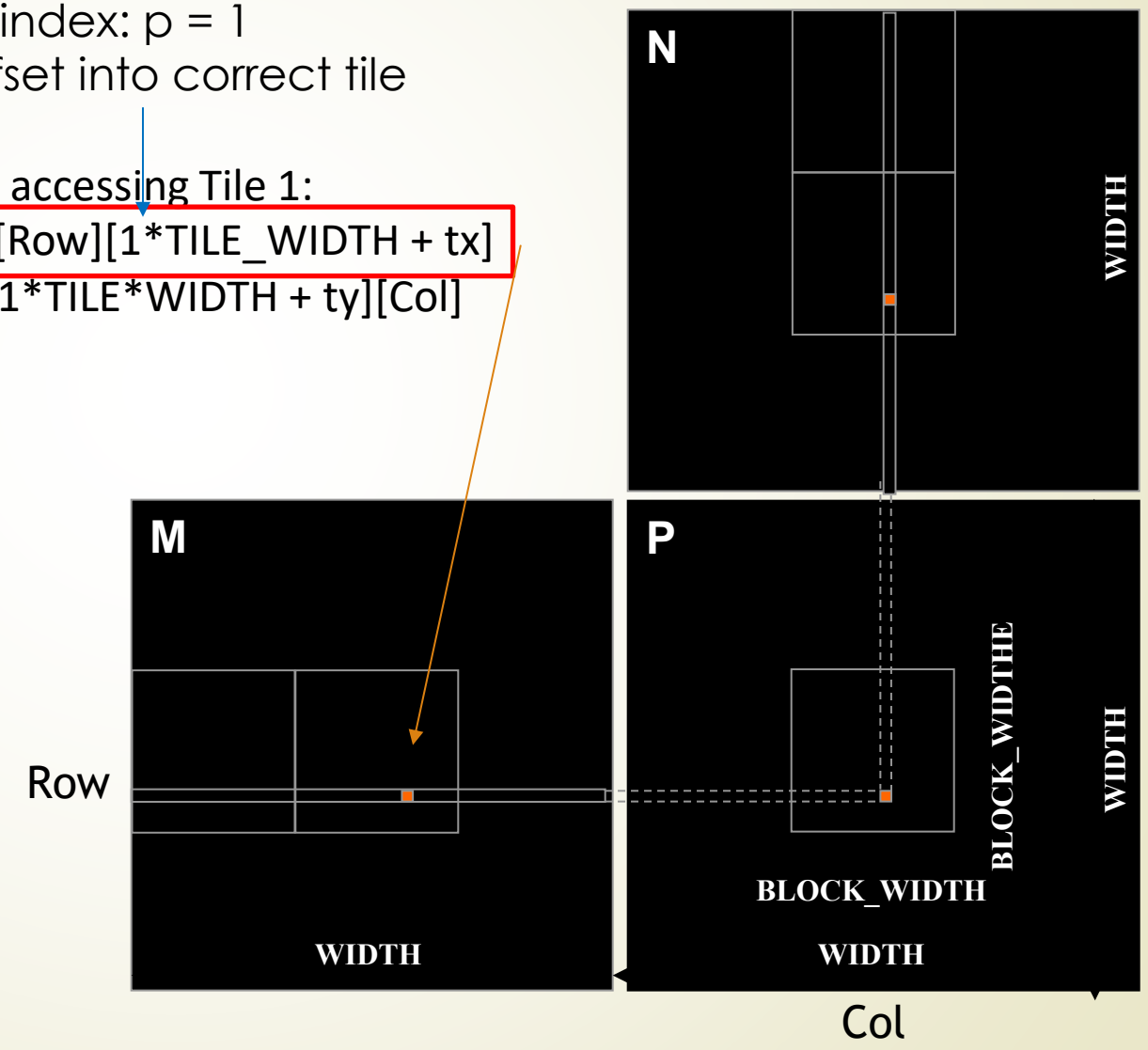
DEFINE phase index: p = 1

- provides offset into correct tile

2D indexing for accessing Tile 1:

M[Row][1*TILE_WIDTH + tx]

N[1*TILE*WIDTH + ty][Col]

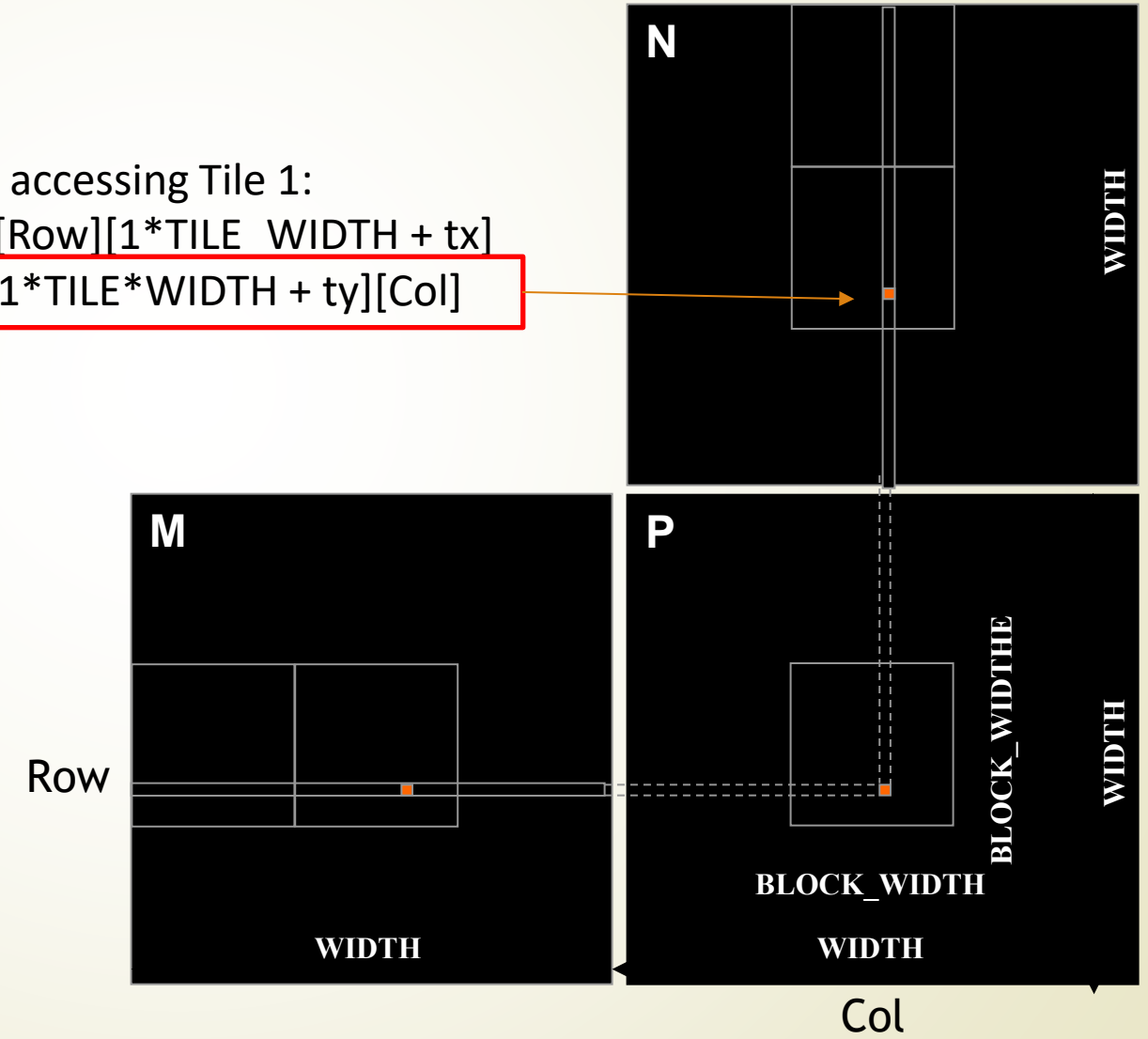


Loading Input Tile 1 of N (Phase 1)

2D indexing for accessing Tile 1:

M[Row][1*TILE_WIDTH + tx]

N[1*TILE*WIDTH + ty][Col]



BUT… M and N are dynamically allocated

And transferred from host->device as linear memory arrays

Thus we must use 1D array indexing inside the CUDA Kernel

Flattening indices into 1D gives:

M[Row][p*TILE_WIDTH+tx]
⇨ M[Row*Width + p*TILE_WIDTH + tx]

N[p*TILE_WIDTH+ty][Col]
⇨ N[(p*TILE_WIDTH+ty)*Width + Col]

where p is the sequence number of the current **phase**

Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, Int Width)
{
  __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
  __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

  int bx = blockIdx.x;  int by = blockIdx.y;
  int tx = threadIdx.x; int ty = threadIdx.y;

  int Row = by * blockDim.y + ty;
  int Col = bx * blockDim.x + tx;
  float Pvalue = 0;

 // Loop over the M and N tiles required to compute the P element
 for (int p = 0; p < n/TILE_WIDTH; ++p) {
    // LOAD: Collaborative loading of M and N tiles into shared memory
    ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
    ds_N[ty][tx] = N[(t*TILE_WIDTH+ty)*Width + Col];
    __syncthreads();

    // USE: compute partial-dot-product using current SMEM tile values
    for (int i = 0; i < TILE_WIDTH; ++i)Pvalue += ds_M[ty][i] * ds_N[i][tx];
    __synchthreads();
  }
  P[Row*Width+Col] = Pvalue;
}
```

Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, Int Width)
{
  __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
  __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

  int bx = blockIdx.x;  int by = blockIdx.y;
  int tx = threadIdx.x; int ty = threadIdx.y;

  int Row = by * blockDim.y + ty;
  int Col = bx * blockDim.x + tx;
  float Pvalue = 0;

 // Loop over the M and N tiles required to compute the P element
 for (int p = 0; p < n/TILE_WIDTH; ++p) {
    // LOAD: Collaborative loading of M and N tiles into shared memory
    ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
    ds_N[ty][tx] = N[(t*TILE_WIDTH+ty)*Width + Col];
    __syncthreads();

    // USE: compute partial-dot-product using current SMEM tile values
    for (int i = 0; i < TILE_WIDTH; ++i)
        Pvalue += ds_M[ty][i] * ds_N[i][tx];
    __synchthreads();
  }
  P[Row*Width+Col] = Pvalue;
}
```

Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, Int Width)
{
  __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
  __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

  int bx = blockIdx.x;  int by = blockIdx.y;
  int tx = threadIdx.x; int ty = threadIdx.y;

  int Row = by * blockDim.y + ty;
  int Col = bx * blockDim.x + tx;
  float Pvalue = 0;

 // Loop over the M and N tiles required to compute the P element
 for (int p = 0; p < n/TILE_WIDTH; ++p) {
    // LOAD: Collaborative loading of M and N tiles into shared memory
    ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
    ds_N[ty][tx] = N[(t*TILE_WIDTH+ty)*Width + Col];
    __syncthreads();

    // USE: compute partial-dot-product using current SMEM tile values
    for (int i = 0; i < TILE_WIDTH; ++i)
        Pvalue += ds_M[ty][i] * ds_N[i][tx];
    __synchthreads();
  }
  P[Row*Width+Col] = Pvalue;
}
```

Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, Int Width)
{
  __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
  __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

  int bx = blockIdx.x;  int by = blockIdx.y;
  int tx = threadIdx.x; int ty = threadIdx.y;

  int Row = by * blockDim.y + ty;
  int Col = bx * blockDim.x + tx;
  float Pvalue = 0;

 // Loop over the M and N tiles required to compute the P element
 for (int p = 0; p < n/TILE_WIDTH; ++p) {
    // LOAD: Collaborative loading of M and N tiles into shared memory
    ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
    ds_N[ty][tx] = N[(t*TILE_WIDTH+ty)*Width + Col];
    __syncthreads();

    // USE: accumulate partial-dot-product using current SMEM tile values
    for (int i = 0; i < TILE_WIDTH; ++i)
        Pvalue += ds_M[ty][i] * ds_N[i][tx];
    __synchthreads();
  }
  P[Row*Width+Col] = Pvalue;
}
```

Tiled Matrix Multiplication Kernel

```
__global__ void MatrixMulKernel(float* M, float* N, float* P, Int Width)
{
  __shared__ float ds_M[TILE_WIDTH][TILE_WIDTH];
  __shared__ float ds_N[TILE_WIDTH][TILE_WIDTH];

  int bx = blockIdx.x;  int by = blockIdx.y;
  int tx = threadIdx.x; int ty = threadIdx.y;

  int Row = by * blockDim.y + ty;
  int Col = bx * blockDim.x + tx;
  float Pvalue = 0;

 // Loop over the M and N tiles required to compute the P element
 for (int p = 0; p < n/TILE_WIDTH; ++p) {
    // LOAD: Collaborative loading of M and N tiles into shared memory
    ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH+tx];
    ds_N[ty][tx] = N[(t*TILE_WIDTH+ty)*Width + Col];
    __syncthreads();

    // USE: accumulate partial-dot-product using current SMEM tile values
    for (int i = 0; i < TILE_WIDTH; ++i)
        Pvalue += ds_M[ty][i] * ds_N[i][tx];
    __synchthreads();
  }
  P[Row*Width+Col] = Pvalue; // write final complete result to GMEM
}
```

Tiled-MMULT Arithmetic Intensity Analysis

- Tiled MMULT reduces GMEM accesses by factor TILE_WIDTH
- Thus the AI improves!
  - Naïve MMULT kernel was 2 FLOP / 8 Bytes = 0.25 FLOP/B (for FP32)
  - Tiled MMULT is TILE_WIDTH * 2 FLOP / 8 Byte = TILE_WIDTH / 4
    - TILE_WIDTH = 16 : AI = 4
    - TILE_WIDTH = 32 : AI = 8
    - TILE_WIDTH = 128 : AI = 32 * Available Shared Memory / SM is limiter!
- H100 peak single-precision (FP32) performance
  - (66.9 TFLOPS) / (3.35 TB/s) = 19.97 FLOP/B ~ 20 FLOP/B

HW2 Overview

DUE: 27 October 2026, 5:59 PM

Learning Objectives

1. Practice applying parallel performance theory concepts
2. Implement 2D gaussian blur CUDA kernel variants
3. Use device shared memory to store and access the blur filter coefficients
4. Use Nsight VSE CUDA debugger to inspect breakpoints in kernel execution threads
5. Use Nsight Compute Command-Line Interface (CLI) to profile kernel execution
6. Use Host timing functions to measure host and device runtimes
7. Compare theoretical performance metric estimates to empirical measured values

- You should have sufficient information to complete this assignment using the contents of lecture 3 and reading set #3.
- NOTE: submitting HW on Canvas – what not to include…

Profiling Host Applications with Windows High-Performance Timer API

- #include "profileapi.h"
  - On AppStream VDE in
  - C:\Program Files (x86)\Windows Kits\10\Include\10.0.26100.0\um
- https://learn.microsoft.com/en-us/windows/win32/api/profileapi/

QueryPerformanceCounter( )

- Retrieves the current value of the performance counter, which is a high resolution (<1us) time stamp that can be used for time-interval measurements.
- https://learn.microsoft.com/en-us/windows/win32/api/profileapi/nf-profileapi-queryperformancecounter

QueryPerformanceFrequency( )

- Retrieves the frequency of the performance counter.
- https://learn.microsoft.com/en-us/windows/win32/api/profileapi/nf-profileapi-queryperformancefrequency

You'll use this in HW2 procedure section "**Host-side Code Execution Timing**"